



DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Robotics, Cognition, Intelligence

RNNs with Independency Assumptions: Scalable and Efficient Sequence Learning

Shaofei Liu





DEPARTMENT OF INFORMATICS

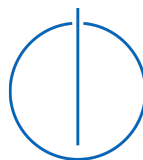
TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Robotics, Cognition, Intelligence

RNNs with Independency Assumptions: Scalable and Efficient Sequence Learning

RNNs mit Unabhängigkeits-Annahme: Skalierbares und Effizientes Sequenzlernen

Author:	Shaofei Liu
Supervisor:	Prof. Dr. Daniel Cremers
Advisor:	Yuesong Shen
Submission Date:	15.01.2022



I confirm that this master's thesis in robotics, cognition, intelligence is my own work and I have documented all sources and material used.

Munich, 15.01.2022

Shaofei Liu

Acknowledgments

I am indebted to my advisor, Yuesong Shen, for his continuous guidance in this project over the past six months. He gave me a lot of help and advice with patience, meticulousness and enthusiasm. I have benefited greatly from his profound knowledge and keen insight in computer science and deep learning.

I would also like to express my gratitude to my thesis supervisor, Prof. Dr. Daniel Cremers. This research work would not have been possible without his help and support.

Lastly, my sincere thanks go to my parents. Their continued support in this epidemic period has allowed me to always focus on the project.

Abstract

A recurrent neural network or RNN is a type of artificial neural network in which all recurrent units are connected in a chain. It can learn nonlinear features in input sequences. Traditional fully connected RNNs often encounter exploding and vanishing gradient problems, so the IndRNN proposes to endow independency to RNNs, i.e., each hidden state neuron in one layer is independent of others and learns one sort of sequential pattern. This structure makes the training of RNNs more stable. However, the connections between the neurons in the original IndRNN are too simplified, which is not conducive to expressing complex hidden features and also weakens temporal dependencies in a long sequence. This paper proposes a new type of RNN, the IndRevRNN, which brings the idea of RevNet into IndRNN. This new model aims to better characterize complex feature correlations between two adjacent time steps in a scalable sequence with reversible residual blocks. It also addresses the problem of long-term dependencies through the memory capacity of hidden state vectors and cell state vectors in the model. Since the IndRevRNN employs parallel computing on GPUs with CUDA and has a reversible structure, it is computationally and memory efficient. Experiments show that this new RNN structure performs on par with or better than state-of-the-art methods on specific tasks.

Contents

Acknowledgments	iii
Abstract	iv
1 Introduction	1
1.1 Topic and motivation of the research	1
1.2 Content outline	3
2 Related work	4
2.1 Early and recent RNN structures	4
2.2 Regularization methods	7
2.3 Normalization approaches	9
3 Theoretical background	13
3.1 The vanilla RNN	13
3.2 The IndRNN and its independencies	15
3.3 The RevNet and reversible residual blocks	17
4 RNNs with independency assumptions	20
4.1 The IndLSTM network	20
4.2 Basic IndRevRNN	23
4.3 Using cell factors to regulate long-term memory	28
4.4 Using forget gates as trainable information control units	30
5 Experiments and results	34
5.1 Parameter initialization	34
5.2 Experiment results on the MNIST dataset	36
5.3 Experiment results on the Penn Treebank corpus	41
6 Conclusion and future work	47
6.1 Conclusion and discussions	47
6.2 Future work	48

A Implementation of the IndRevRNN algorithm	49
A.1 Python baseline of IndRevRNN	49
A.2 Custom CUDA implementation of IndRevRNN	54
A.3 IndRevRNN class module and parameter initialization	66
List of Figures	69
List of Tables	71
Bibliography	71

1 Introduction

To better understand the context in which the research of this thesis belongs, this chapter will briefly introduce the topic and motivation of the work. Also, an outline of the contents in other chapters will be given.

1.1 Topic and motivation of the research

Many learning tasks require processing sequential data. Since Rumelhart et al. proposed the concept of backpropagation in 1986, recurrent neural networks or RNNs have become a vital way for learning features in sequences [69]. Common scenarios for RNNs include machine translation [18], speech recognition [62][71], sequence data prediction [73][78][2], and gratifying achievements have been made in these areas. Similar to convolutional neural networks or CNNs that can learn features in images of different sizes [4], RNNs can efficiently process patterns in long, scalable and even infinite sequences, once enough parameters are used in the model.

RNNs are theoretically Turing complete and have a parameter sharing scheme [41]. Unlike standard feedforward neural networks, recurrent networks retain memory states that can represent information from arbitrarily long sequences [58]. The core part of an RNN is a directed graph. Elements connected in chains in the graph are called recurrent units, as shown in Figure 1.1.

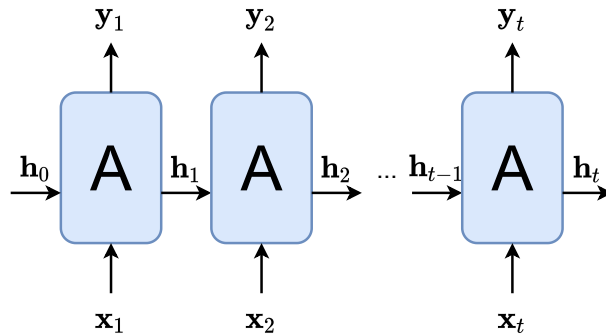


Figure 1.1: Basic structure and recurrent units of an RNN.

The vanilla RNN is known to be susceptible to exploding and vanishing gradient problems [38][71], which is difficult to regulate due to the matrix products in gradient descent and backpropagation [66]. Network structures such as LSTM and GRU and their variants are proposed to solve the gradient problems [39][10]. However, these networks tend to have more parameters and more complex structures than the vanilla RNN, thus requiring a lot of storage and computation resources.

To restrict model parameters within a certain range and avoid gradient problems, the IndRNN proposes to replace some matrix multiplications between weights and data with element-wise multiplications, which allows the computation of each hidden neuron to be independent [57]. This approach simplifies the operations at each time step and also makes it possible to build deeper networks.

However, the connections between hidden states and input data in the original IndRNN are too simple to learn complex pattern correspondences between a sequence's preceding and following items. This model also lacks a component that simulates long-term memory, making it difficult to represent the information overlap between two far apart elements in the same array.

To address the issues of other models discussed above, a new RNN architecture, the independently reversible residual recurrent neural network or the IndRevRNN, is proposed in this thesis. The IndRevRNN introduces reversible residual blocks inspired by RevNet into IndRNN [29], which has the advantages of both.

In the reversible residual blocks of IndRevRNN, the weight vectors with nonlinear activation functions can learn complex nonlinear features between two adjacent time steps in a sequence. The learned information is passed from the front end to the back end of a sequence between the cell state vectors, which simulate long-term memory, and the hidden state vectors, which simulate short-term memory in the reversible residual blocks. With this structure, even if a sequence is very long or scalable, valid information can still be retained across multiple time steps without excessive loss.

The reversible structure in IndRevRNN enables intermediate variables to be computed from both the input and output of a residual block, which reduces the need for computation and storage. The IndRevRNN is also implemented through parallel programming on the CUDA platform, so the programs run with high efficiency.

To sum up, as a new RNN structure, the IndRevRNN avoids the exploding and vanishing gradient problems, can learn complex feature correlations between two adjacent inputs of a long or scalable sequence, tackles the problem of long-term dependencies with the information flow between the hidden states and cell states, does not require excessive computation and storage for intermediate variables, and also takes advantage of high speed parallel computing.

Experiments show that the IndRevRNN performs comparable or better than state-of-the-art methods on the permuted sequential MNIST dataset and Penn Treebank corpus.

It can also be easily applied to other sequence learning scenarios.

1.2 Content outline

This chapter introduced the background and motivation for designing the IndRevRNN and explained the features and advantages of this new architecture.

In chapter 2, some work related to IndRevRNN with their inspiration for the network design in this thesis are discussed.

In chapter 3, the theoretical background of IndRevRNN and the source of ideas in designing different model parts are introduced.

In chapter 4, the basic network structure and different variants of IndRevRNN are shown.

In chapter 5, the parameter initialization in IndRevRNN is discussed. The experiments and results using different model variants on the permuted sequential MNIST dataset and Penn Treebank corpus are also reviewed.

In chapter 6, the whole thesis is summarized and some possible future work on IndRevRNN are proposed.

2 Related work

The previous work related to the new network structure proposed in this thesis, the IndRevRNN, is mainly included in the following three categories: other RNN structures with similar sequence learning functions, regularization methods to reduce overfitting and normalization regimes to stabilize or accelerate training. This chapter will introduce some related researches from the above three aspects and their inspiration for the designs in this thesis.

2.1 Early and recent RNN structures

The long short-term memory or LSTM network proposed by Hochreiter et al. in 1997 is a widely used variant of RNN [39]. This structure is frequently used in many fields such as language modeling [81], traffic flow analysis [25], trajectory prediction [3] and even infectious disease research [17].

The purpose of its invention is to solve the vanishing gradient problems. The key insight in the design of LSTM is to incorporate nonlinear controls trained from the data into an RNN unit, which ensures the gradient of the objective function relative to various parameters in a long sequence will not disappear [77]. Figure 2.1 shows a canonical LSTM network.

In an LSTM network, the information in the cell states only participates in some minor linear interactions, so it can be used to simulate the characteristics of long-term memory. Using the structure called gates, which selectively let information through, LSTM can delete or add information to the cell states [65].

The first step of LSTM is to decide what information to discard from the cell states, which is selected by the forget gate $\mathbf{f}_t$. The next step is to decide what new information to store in the cell states, which is controlled by the input gate $\mathbf{i}_t$ and $\tilde{\mathbf{c}}_t$. The last step is to decide what information to output based on a filtered version of cell states, which is handled by the output gate $\mathbf{o}_t$. Researches show that the forget gates are the most important part of LSTM networks in performance improvement, followed by input gates and output gates [43][31].

LSTM offers two inspirations for the network structure proposed in this thesis. One is that adding cell states that only change slightly in different time steps to the model is helpful to tackle long-term dependencies. The other is that the gate modules added to

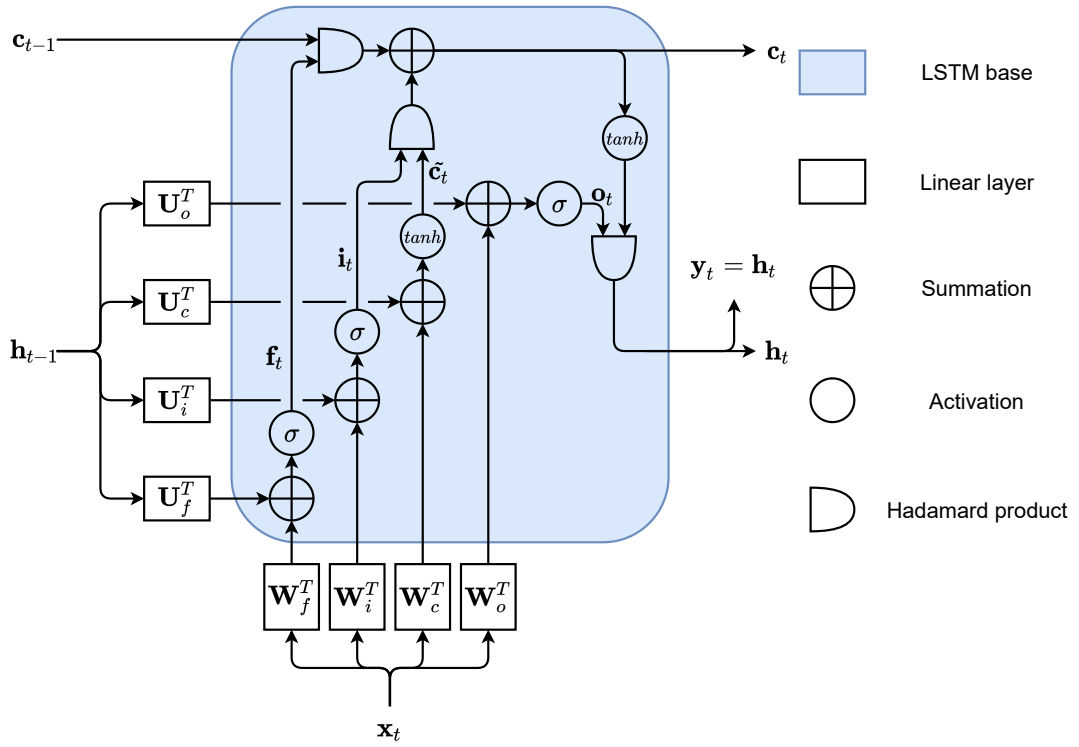


Figure 2.1: The LSTM network.

the network are equivalent to trainable information control units. Both of these ideas are considered in the design of IndRevRNN.

The LSTM network in Figure 2.1 and some of its close variants, such as the Peephole LSTM network in [27], contain more model parameters and are more laborious in computations than a vanilla RNN that will be discussed in chapter 3. An architecture similar to LSTM with fewer model parameters is the gated recurrent unit or GRU [18], as shown in Figure 2.2.

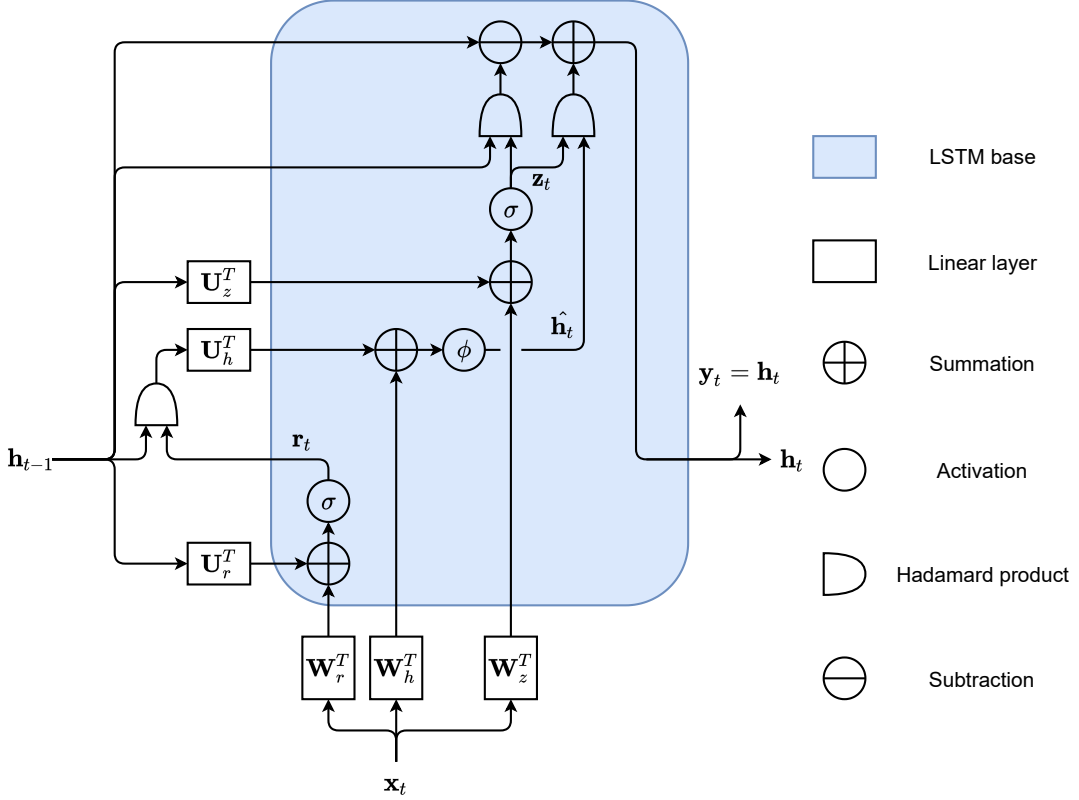


Figure 2.2: The GRU cell.

It is found that GRU networks have similar performances as LSTM networks in the jobs like traffic flow prediction and speech recognition [25][68], while at times tasks with rather small data sets work better on the former ones [32]. Some GRU variants that further improve the applicability have been developed in [22].

In addition to LSTM and GRU, many novel RNN structures have been designed in recent years. The Unitary Evolution Recurrent Neural Networks explore the use of orthogonal matrices and unitary matrices that have eigenvalues of absolute value exactly 1 to avoid

sudden gradient changes, without expensive computations such as eigendecomposition in RNNs [7]. Skip RNN proposes a novel RNN architecture that allows state update omissions and reduces the number of sequential operations without extra supervision signals [15]. AWD-LSTM employs DropConnect for regularization and NT-ASGD for optimization, which exerts advanced potential of LSTM in terms of word-level language modeling [61]. h-DETACH proposes a stochastic algorithm for LSTM optimization that in expectation scales the various gradient components, thereby preventing the gradients through cell state paths from being suppressed [8]. nnRNN proposes a new connection pattern based on Schur decomposition, which retains the stability of orthogonal RNNs while enhancing the expression ability [45].

Although the above work have improved the performance of early RNNs, they still suffer from massive computation and storage. For this, a good suggestion is to exploit the possibility of parallel computing [40], where typical examples are Quasi-Recurrent Neural Networks or QRNN, Kernel Neural Networks or KNN and Simple Recurrent Units or SRU [14][54][55].

QRNN allows parallel computations across batch and time step dimensions, which empowers high throughput and good scalability for long sequences [14]. KNN appeals to the architecture design with kernels over combinatorial structures to define the appropriate computational operations [54]. SRU exhibits the same level of parallelism as convolutional and feedforward networks, which is achieved by balancing sequential dependence and independence with more recurrent connections and highway channels [55].

All these methods make effective use of high-performance computations and have attempted to tackle sequence learning problems. However, they either lack trainable RNN layers or partially ignore the continuity and order conditions in sequences during training. IndRNN [57] and Deep IndRNN [56] have a response to these issues. Their characteristics are discussed in detail in chapter 3.

2.2 Regularization methods

Overfitting is a common problem in neural network training [34], which is manifested in that the accuracy of the model on the training data set is far better than that on the validation and test data sets. Overfitting means that the model contains more parameters than that can be trained by the given valid data, thereby learning the noise in it [24][5].

To reduce the chance or degree of overfitting, many techniques can be used in the training process such as model comparison, cross-validation, regularization, early stopping, pruning, Bayesian prior or dropout [90]. Among them, dropout is an efficient way in neural networks to perform model averaging [37].

The key idea of the original dropout scheme is to randomly drop units along with

their connections from the neural network, preventing units from co-adapting too much, as shown in Figure 2.3. This approach provides a way of approximately combining exponentially many different network architectures for training, which effectively prevents overfitting [79]. However, applying standard dropout to RNNs tends to limit the ability of the networks to retain their memory.

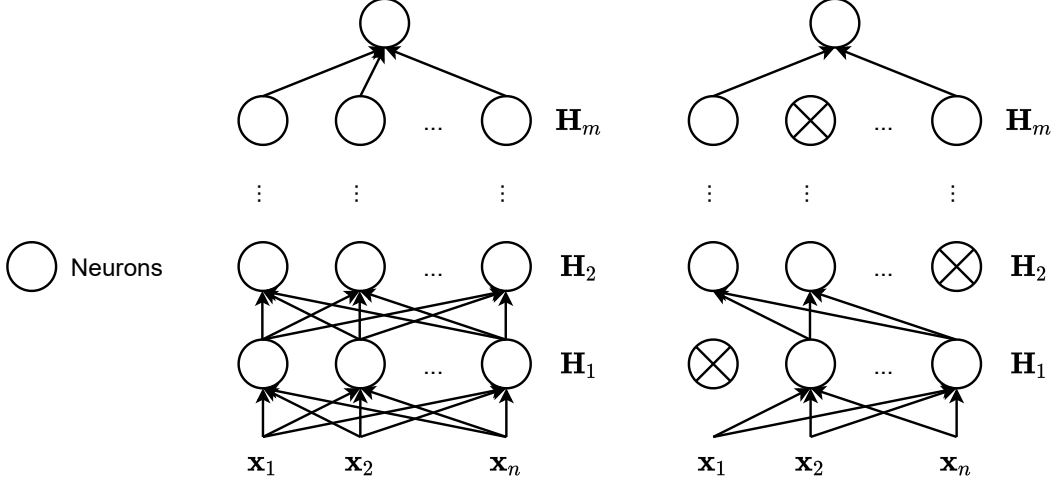


Figure 2.3: The original dropout scheme.

The first applicable dropout method on RNNs is demonstrated in [91], whose arrangement is similar to that in Figure 2.4. The LSTM networks can benefit from this regularization without sacrificing their valuable memorization ability, but no dropout is applied on recurrent connections.

Variational Dropout by Gal and Ghahramani [26] and Recurrent Dropout by Semeniuta et al. [75] are two dropout variants suitable for recurrent connections.

Variational Dropout applies the same dropout mask called a per-sequence dropout mask for inputs, outputs or recurrent layers, which means to drop the same network units at each time step. It also has a specific dropout pattern on word embeddings for natural language processing [26]. Variational Dropout is nevertheless not favorable when considering potential memory loss issues in LSTM architectures [74].

Recurrent Dropout on the contrary employs a per-step dropout mask. It allows to freely apply dropout in the recurrent connections of a gated network without hindering its ability to process long-term relationships [75].

A novel method similar to dropout for regularizing RNNs is Zoneout by Krueger et al. [49]. It randomly replaces some activations of the units with their activations from the previous time step. Zoneout makes it easier for the network to preserve information

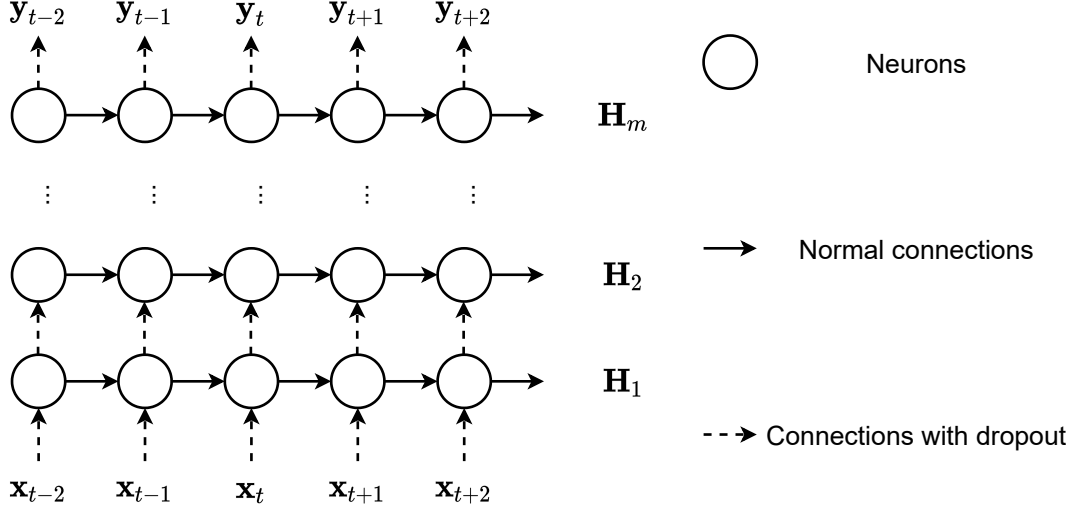


Figure 2.4: First dropout applied to RNNs.

going forward, and facilitates, rather than hinders, the flow of gradient information going backward.

An example of Variational Dropout, Recurrent Dropout and Zoneout used in LSTM networks is shown in Figure 2.5.

The main dropout methods used in the experiments are the ones proposed by [91] and [26] and they are applied to inputs. Other settings are also roughly covered.

2.3 Normalization approaches

Another problem in the training of neural networks is that it is affected by the randomness of parameter initialization and the input data during the training process, which is noted as the internal covariate shift. Batch Normalization normalizes the layer inputs by re-centering and rescaling, making the training of networks faster and more stable [42]. [19] further develops the recurrent version of Batch Normalization. An example of using Batch Normalization in RNNs is shown in Figure 2.6a.

Batch Normalization helps the network to produce better gradients for weight updates as well as provides a more stable and accelerated training regime [19]. However, it requires rather large batch sizes during training and adds additional complexity in RNNs if used between time steps, i.e., the independency assumptions no longer hold.

Layer Normalization is proposed to resolve the former matter, which removes the dependency on batches [9]. It normalizes the activations along the feature direction

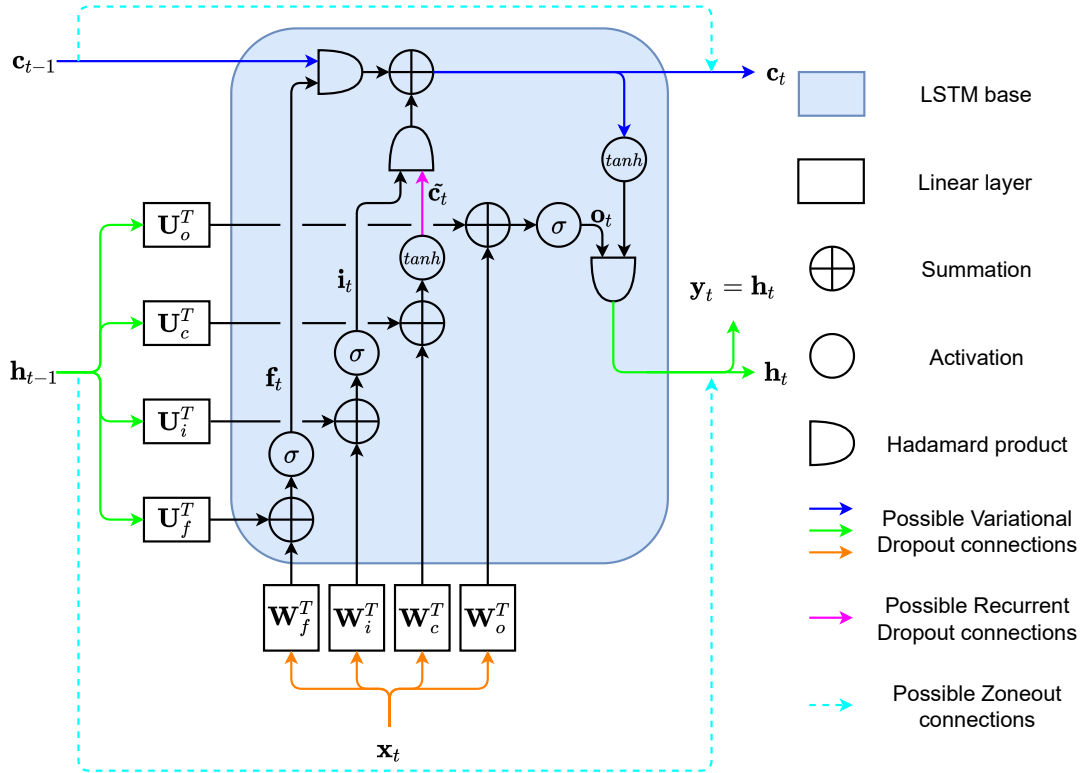


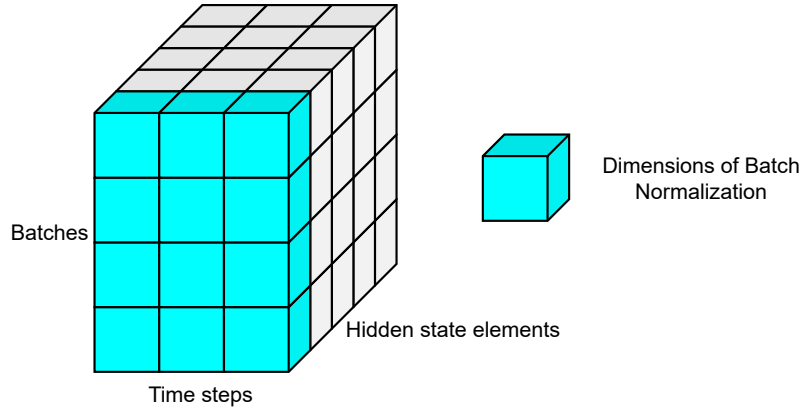
Figure 2.5: Dropout examples in an LSTM cell.

instead of the mini-batch direction, as shown in Figure 2.6b. Same as Batch Normalization, it also violates the independency assumptions and is not simply applicative on parallel recurrent connections.

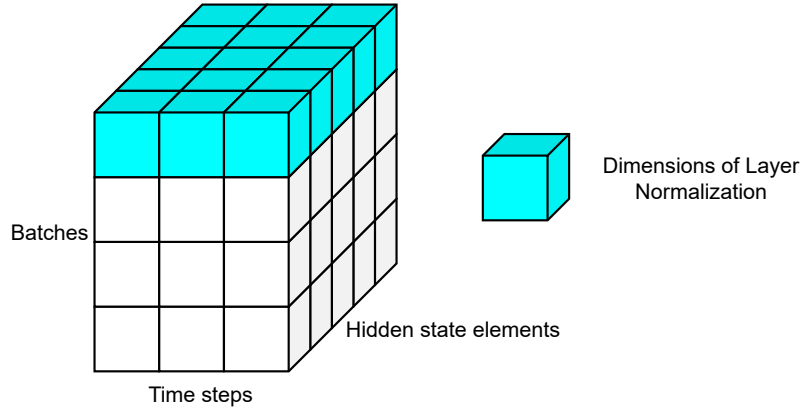
Another normalization variant is Weight Normalization documented in [72], whose aim is to improve the conditioning of optimization problems and to speed up the convergence of stochastic gradient descent. Experiments prove that they lead to less stable networks compared to Batch Normalization and therefore their popularization is limited.

Group Normalization lies in the middle of Layer Normalization and Instance Normalization in [82], as shown in Figure 2.6c. It divides the features into certain groups and normalizes each group separately, hence is more flexible than Layer Normalization [87]. This concept is nonetheless originally designed for CNNs, and there are few examples of its use on RNNs.

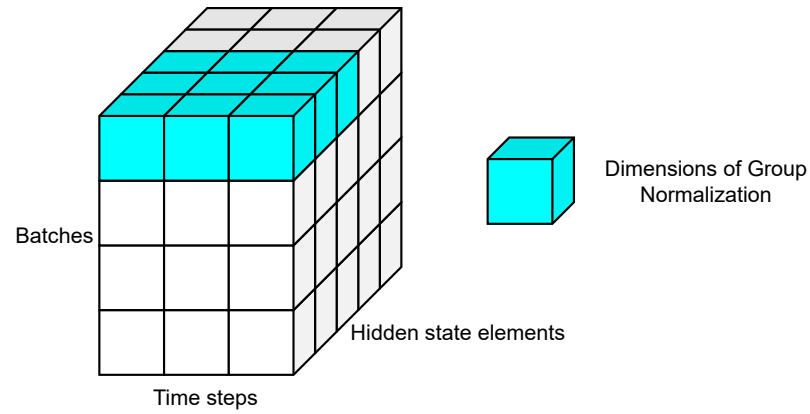
To ensure independence, the normalization approaches used in the experiments are mainly Batch Normalization [42] and its variant for recurrent networks [19]. Other normalization methods have also undergone some attempts.



(a)



(b)



(c)

Figure 2.6: Dimensions of Batch Normalization, Layer Normalization and Group Normalization in an RNN.

3 Theoretical background

This chapter will introduce the vanilla RNN, IndRNN and RevNet, which are closely related to IndRevRNN. The features and computation processes of these previous networks will be analyzed. This forms the computational and theoretical basis for the design of IndRevRNN.

3.1 The vanilla RNN

The structure of a commonly used single-layer multi-step RNN, the vanilla RNN, is shown in Figure 3.1. In the network, $\mathbf{x}_t \in \mathbb{R}^n$ denotes the input vector at time step t . $\mathbf{h}_t \in \mathbb{R}^m$ represents the hidden state vector involved in the operation at time step $t + 1$, which is also equal to the output vector $\mathbf{y}_t \in \mathbb{R}^m$ at time step t . $\mathbf{h}_0 \in \mathbb{R}^m$ stands for the experience and task specified prior hidden state vector. $\mathbf{W}^T \in \mathbb{R}^{m \times n}$ and $\mathbf{U}^T \in \mathbb{R}^{m \times m}$ signify the weight matrices that are multiplied by $\mathbf{x}_t$ and $\mathbf{h}_t$, respectively. At each time step, two matrix-vector products are summed and substituted in a nonlinear activation function to get the output vector $\mathbf{y}_t$.

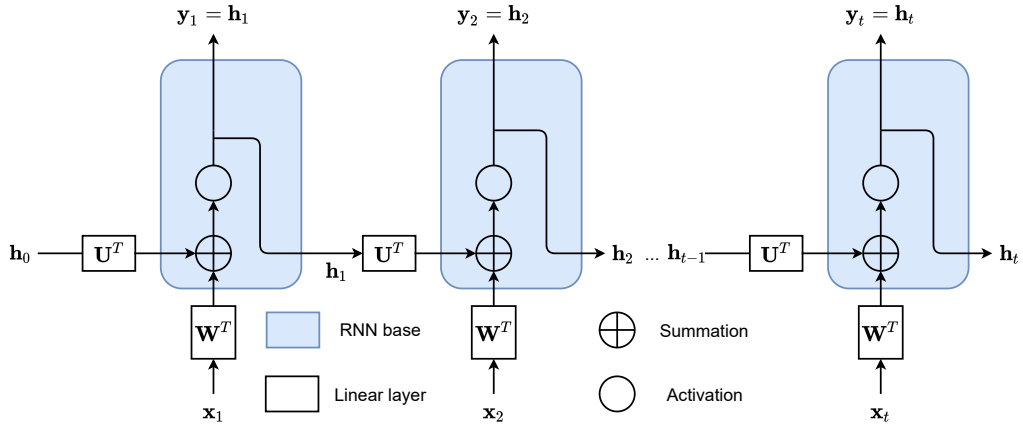


Figure 3.1: A typical vanilla RNN structure.

Since each time step t shares the same weight matrices $\mathbf{W}^T$ and $\mathbf{U}^T$, the RNN model in Figure 3.1 can be simplified to an RNN cell in Figure 3.2. The update rule of the

output vector or the hidden state vector at each time step t is expressed by Equation 3.1, where the bias term of the linear layer is omitted.

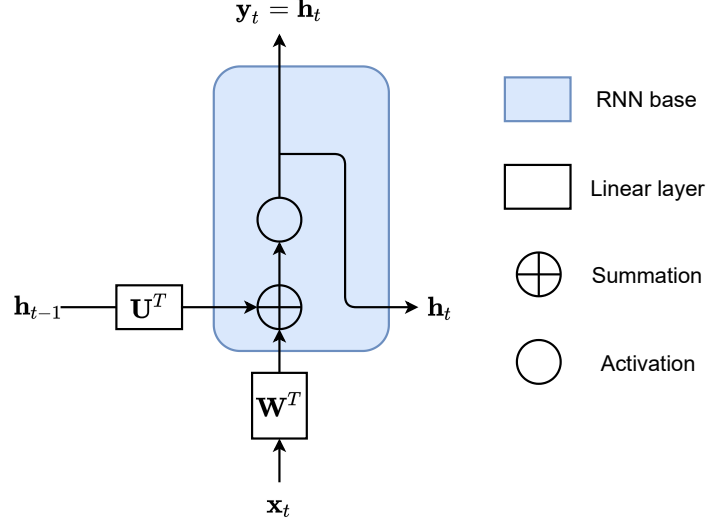


Figure 3.2: The vanilla RNN cell.

$$\mathbf{y}_t = \mathbf{h}_t = \sigma(\mathbf{W}^T \mathbf{x}_t + \mathbf{U}^T \mathbf{h}_{t-1}) \quad (3.1)$$

In the forward pass, the output vectors $\mathbf{y}$ of an RNN layer will be used as input for other network components according to task specifications. For example, these outputs can be combined with a linear layer and a softmax function to get a classifier output in multinomial logistic regression.

In the backward pass, let $\mathcal{L}$ be the loss function of the forward pass for simplicity. One usually obtains the partial derivative of $\mathcal{L}$ with respect to $\mathbf{y}$ from other parts of the network, which is $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ [16]. The complete error backpropagation in the backward pass is shown in Equation 3.2, Equation 3.3, Equation 3.4 and Equation 3.5, where $\circ$ represents the Hadamard product.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{W}^T} &= \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{W}^T} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}^T} \\ &= \sum_t \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right) \circ \sigma'(\mathbf{W}^T \mathbf{x}_t + \mathbf{U}^T \mathbf{h}_{t-1}) \cdot \mathbf{x}_t^T \end{aligned} \quad (3.2)$$

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial \mathbf{U}^T} &= \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{U}^T} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{U}^T} \\
 &= \sum_t \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right) \circ \sigma'(\mathbf{W}^T \mathbf{x}_t + \mathbf{U}^T \mathbf{h}_{t-1}) \cdot \mathbf{h}_{t-1}^T
 \end{aligned} \tag{3.3}$$

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial \mathbf{x}_t} &= \frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{x}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{x}_t} \\
 &= \mathbf{W} \cdot \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right) \circ \sigma'(\mathbf{W}^T \mathbf{x}_t + \mathbf{U}^T \mathbf{h}_{t-1})
 \end{aligned} \tag{3.4}$$

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-1}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_{t-1}} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \\
 &= \mathbf{U} \cdot \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right) \circ \sigma'(\mathbf{W}^T \mathbf{x}_t + \mathbf{U}^T \mathbf{h}_{t-1})
 \end{aligned} \tag{3.5}$$

3.2 The IndRNN and its independencies

As discussed in chapter 1, the vanilla RNN structure in Figure 3.2 often encounters exploding and vanishing gradient problems. An intuitive solution is to simplify some weight matrices in the network to vectors so that each element within can be regulated into a certain range. The independently recurrent neural network or IndRNN proposes to replace the weight matrix $\mathbf{U}^T$ in the forward pass with a weight vector $\mathbf{u} \in \mathbb{R}^m$, and replace the matrix-vector multiplication between $\mathbf{U}^T$ and $\mathbf{h}_t$ with an element-wise product [57]. The cell structure of an IndRNN and the update rule at each time step t are shown in Figure 3.3 and Equation 3.6.

$$\mathbf{y}_t = \mathbf{h}_t = \sigma(\mathbf{W}^T \mathbf{x}_t + \mathbf{u} \circ \mathbf{h}_{t-1}) \tag{3.6}$$

Since the product between the hidden state vector $\mathbf{h}_t$ and the weight vector $\mathbf{u}$ in Equation 3.6 is a Hadamard product, each entry in $\mathbf{h}_t$ can be computed independently, as shown in Equation 3.7. This is also the source of the name IndRNN. k is the index of the element participating in the operation in each vector, for example, u^k stands for the k -th element in $\mathbf{u}$. $\mathbf{w}^{kT}$ denotes the k -th row of the weight matrix $\mathbf{W}^T$.

$$y_t^k = h_t^k = \sigma(\mathbf{w}^{kT} \mathbf{x}_t + u^k h_{t-1}^k) \tag{3.7}$$

Equation 3.8, Equation 3.9, Equation 3.10 and Equation 3.11 list the error backpropagation of an IndRNN. It can be seen from Equation 3.11 that the gradient of h_{t-1}^k directly

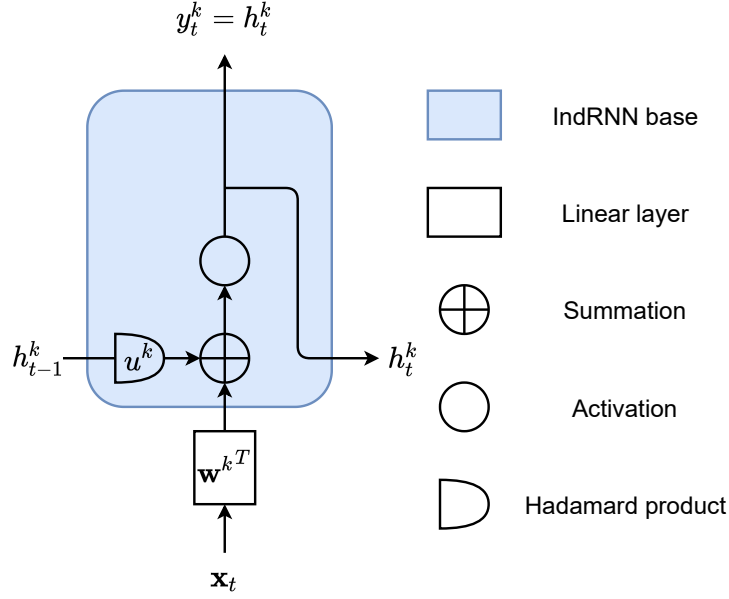


Figure 3.3: Connections of independent neurons in IndRNN.

depends on u^k and $\sigma'(\mathbf{w}^k{}^T \mathbf{x}_t + u^k h_{t-1}^k)$ with respect to the gradient of h_t^k , where u^k is changed by a small magnitude according to the learning rate and $\frac{\partial \mathcal{L}}{\partial u^k}$. Thus regulating the term $u^k \cdot \sigma'(\mathbf{w}^k{}^T \mathbf{x}_t + u^k h_{t-1}^k)$ to an appropriate range will make the training of IndRNN more robust and effectively solve the gradient problems [57].

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{w}^k{}^T} &= \sum_t \frac{\partial \mathcal{L}}{\partial y_t^k} \frac{\partial y_t^k}{\partial \mathbf{w}^k{}^T} + \frac{\partial \mathcal{L}}{\partial h_t^k} \frac{\partial h_t^k}{\partial \mathbf{w}^k{}^T} \\ &= \sum_t \left(\frac{\partial \mathcal{L}}{\partial y_t^k} + \frac{\partial \mathcal{L}}{\partial h_t^k} \right) \cdot \sigma'(\mathbf{w}^k{}^T \mathbf{x}_t + u^k h_{t-1}^k) \cdot \mathbf{x}_t^T \end{aligned} \quad (3.8)$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial u^k} &= \sum_t \frac{\partial \mathcal{L}}{\partial y_t^k} \frac{\partial y_t^k}{\partial u^k} + \frac{\partial \mathcal{L}}{\partial h_t^k} \frac{\partial h_t^k}{\partial u^k} \\ &= \sum_t \left(\frac{\partial \mathcal{L}}{\partial y_t^k} + \frac{\partial \mathcal{L}}{\partial h_t^k} \right) \cdot \sigma'(\mathbf{w}^k{}^T \mathbf{x}_t + u^k h_{t-1}^k) \cdot h_{t-1}^k \end{aligned} \quad (3.9)$$

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial \mathbf{x}_t} &= \frac{\partial \mathcal{L}}{\partial y_t^k} \frac{\partial y_t^k}{\partial \mathbf{x}_t} + \frac{\partial \mathcal{L}}{\partial h_t^k} \frac{\partial h_t^k}{\partial \mathbf{x}_t} \\
 &= \mathbf{w}^k \cdot \left(\frac{\partial \mathcal{L}}{\partial y_t^k} + \frac{\partial \mathcal{L}}{\partial h_t^k} \right) \cdot \sigma'(\mathbf{w}^{kT} \mathbf{x}_t + u^k h_{t-1}^k)
 \end{aligned} \tag{3.10}$$

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial h_{t-1}^k} &= \frac{\partial \mathcal{L}}{\partial y_t^k} \frac{\partial y_t^k}{\partial h_{t-1}^k} + \frac{\partial \mathcal{L}}{\partial h_t^k} \frac{\partial h_t^k}{\partial h_{t-1}^k} \\
 &= u^k \cdot \left(\frac{\partial \mathcal{L}}{\partial y_t^k} + \frac{\partial \mathcal{L}}{\partial h_t^k} \right) \cdot \sigma'(\mathbf{w}^{kT} \mathbf{x}_t + u^k h_{t-1}^k)
 \end{aligned} \tag{3.11}$$

Another advantage of IndRNN is that parallel programming can be used for high-performance scientific computing [59]. At time step t , the k -th element of each vector, such as h_t^k , is only related to the k -th element of other vectors, such as u_t^k , and is not relevant to other elements except the k -th of each vector, such as h_t^{k-2} and u_t^{k-2} . Thus the computation with respect to each k -th element can be performed independently of others. For example, h_t^k and h_{t-2}^{k-2} can be involved in the computation simultaneously.

Despite the advantages, there are also some problems in IndRNN in practice. For instance, in each layer, the hidden state vector $\mathbf{h}$ at time step $t - 1$ is only multiplied by a single weight vector $\mathbf{u}$ shared across all time steps before being added to the input after linear layer $\mathbf{w}^{kT} \mathbf{x}$ at time step t , which means the connections of the neurons between neighboring time steps is too simple and is not conducive to expressing complex relationships between adjacent elements of a sequence.

Additionally, the hidden state vector $\mathbf{h}$ frequently participates in various linear and non-linear operations and no component helps the information run through the entire sequence while remaining almost unchanged. This makes the network structure of InnRNN difficult to learn long-term dependencies [13].

To resolve these problems, a new network structure should be designed that more easily handles the correlations between different steps in a long sequence, while maintaining the advantages of parallel computing by not introducing too many model parameters.

3.3 The RevNet and reversible residual blocks

The Reversible Residual Network or RevNet is a variant of ResNets but in some measure reduces memory consumption [29]. A network structure inspired by its method is shown in Figure 3.4.

There can be a total of n reversible residual blocks, each indexed by i . $\mathbf{h}^{i-1} \in \mathbb{R}^m$ and $\mathbf{c}^{i-1} \in \mathbb{R}^m$ denote two output vectors of the $i - 1$ -th block as well as two input vectors of

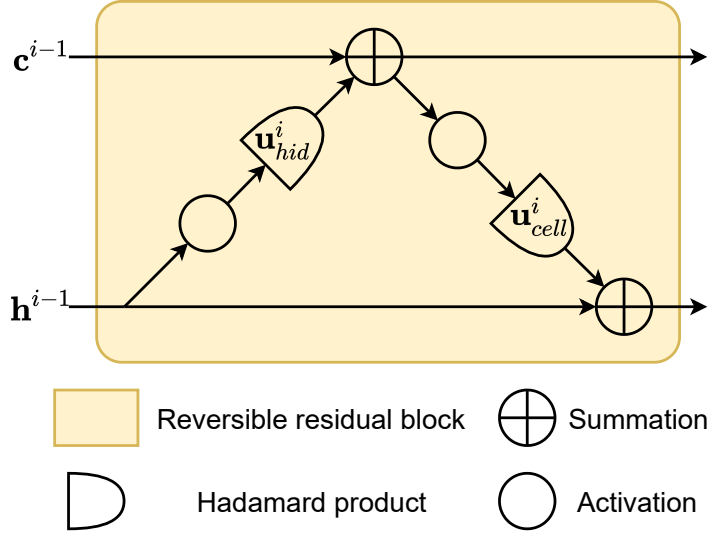


Figure 3.4: A reversible residual block inspired by RevNet.

the i -th block, which are called hidden state vector and cell state vector in this article respectively. $\mathbf{u}_{hid}^i \in \mathbb{R}^m$ and $\mathbf{u}_{cell}^i \in \mathbb{R}^m$ are two weight vectors in the i -th block named hidden weight vector and cell weight vector correspondingly, with which $\mathbf{h}^{i-1}$ and $\mathbf{c}^i$ do an element-wise product after activations.

The forward pass of a reversible residual block is presented by Equation 3.12 and Equation 3.13. Since only additions and entrywise products lie within the computation, the independence of each element in the vectors is still maintained.

$$\mathbf{c}^i = \mathbf{c}^{i-1} + \mathbf{u}_{hid}^i \circ \sigma(\mathbf{h}^{i-1}) \quad (3.12)$$

$$\mathbf{h}^i = \mathbf{h}^{i-1} + \mathbf{u}_{cell}^i \circ \sigma(\mathbf{c}^i) \quad (3.13)$$

Equation 3.14, Equation 3.15, Equation 3.16, Equation 3.17, Equation 3.18 and Equation 3.19 show the backward pass of a reversible residual block. Same as the forward pass, only subtractions and entrywise products are included, which also ensures independence.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{u}_{cell}^i} &= \frac{\partial \mathcal{L}}{\partial \mathbf{h}^i} \frac{\partial \mathbf{h}^i}{\partial \mathbf{u}_{cell}^i} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{h}^i} \circ \sigma(\mathbf{c}^i) \end{aligned} \quad (3.14)$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{c}^i} &= \frac{\partial \mathcal{L}}{\partial \mathbf{h}^i} \frac{\partial \mathbf{h}^i}{\partial \mathbf{c}^i} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{h}^i} \circ \mathbf{u}_{cell}^i \circ \sigma'(\mathbf{c}^i)\end{aligned}\tag{3.15}$$

$$\mathbf{h}^{i-1} = \mathbf{h}^i - \mathbf{u}_{cell}^i \circ \sigma(\mathbf{c}^i)\tag{3.16}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{u}_{hid}^i} &= \frac{\partial \mathcal{L}}{\partial \mathbf{c}^i} \frac{\partial \mathbf{c}^i}{\partial \mathbf{u}_{hid}^i} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{c}^i} \circ \sigma(\mathbf{h}^{i-1})\end{aligned}\tag{3.17}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{i-1}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{c}^i} \frac{\partial \mathbf{c}^i}{\partial \mathbf{h}^{i-1}} \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{c}^i} \circ \mathbf{u}_{hid}^i \circ \sigma'(\mathbf{h}^{i-1})\end{aligned}\tag{3.18}$$

$$\mathbf{c}^{i-1} = \mathbf{c}^i - \mathbf{u}_{hid}^i \circ \sigma(\mathbf{h}^{i-1})\tag{3.19}$$

It is noticeable in Equation 3.16 and Equation 3.19 that in the backward pass, the inputs $\mathbf{h}^{i-1}$ and $\mathbf{c}^{i-1}$ are computed from the outputs $\mathbf{h}^i$ and $\mathbf{c}^i$ of the i -th block. In other words, one only needs to store the outputs $\mathbf{h}^n$ and $\mathbf{c}^n$ of the last reversible residual block in the forward pass, before obtaining $\mathbf{h}^i$ and $\mathbf{c}^i$ of any block i in the backward pass.

The advantage of the reversible residual blocks is that they reduce intermediate variables in the storage while maintaining the connection characteristics of plain residual networks. This feature can be used in the design of IndRevRNN.

4 RNNs with independency assumptions

This chapter will briefly introduce the first network structure designed in the research, the IndLSTM. After that, this chapter will focus on the basic IndRevRNN and some of its variants obtained with small changes in the architecture. A programmatic implementation of the algorithms is presented in Appendix A.

4.1 The IndLSTM network

As mentioned in chapter 3, the new RNN model proposed in this thesis, the IndRevRNN, is inspired by the ideas of IndRNN and RevNet, but this design does not emerge naturally from the beginning. After learning the characteristics of LSTM networks introduced in chapter 2 as well as their rich application scenarios, the first intuitive idea is to introduce parallel computing to an LSTM variant and ameliorate its performance.

It just so happens that the IndRNN makes some minor changes on the basis of a vanilla RNN which can also be similarly applied to an LSTM network. Figure 4.1 shows the result of these modifications, which leads to the so-called Independently LSTM or IndLSTM.

In simple words, compared with the original LSTM network in Figure 2.1, the IndLSTM network in Figure 4.1 replaces four linear weight matrices $\mathbf{U}_o^T$, $\mathbf{U}_c^T$, $\mathbf{U}_i^T$ and $\mathbf{U}_f^T$ on the hidden state vector $\mathbf{h}_{t-1}$ at time step t with four weight vectors $\mathbf{u}_o$, $\mathbf{u}_c$, $\mathbf{u}_i$ and $\mathbf{u}_f$.

The complete feedforward process of IndLSTM is listed by Equation 4.1, Equation 4.2, Equation 4.3, Equation 4.4, Equation 4.5 and Equation 4.6, where the bias terms are also omitted.

$$\mathbf{f}_t = \sigma(\mathbf{W}_f^T \mathbf{x}_t + \mathbf{u}_f \circ \mathbf{h}_{t-1}) \quad (4.1)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{u}_i \circ \mathbf{h}_{t-1}) \quad (4.2)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o^T \mathbf{x}_t + \mathbf{u}_o \circ \mathbf{h}_{t-1}) \quad (4.3)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c^T \mathbf{x}_t + \mathbf{u}_c \circ \mathbf{h}_{t-1}) \quad (4.4)$$

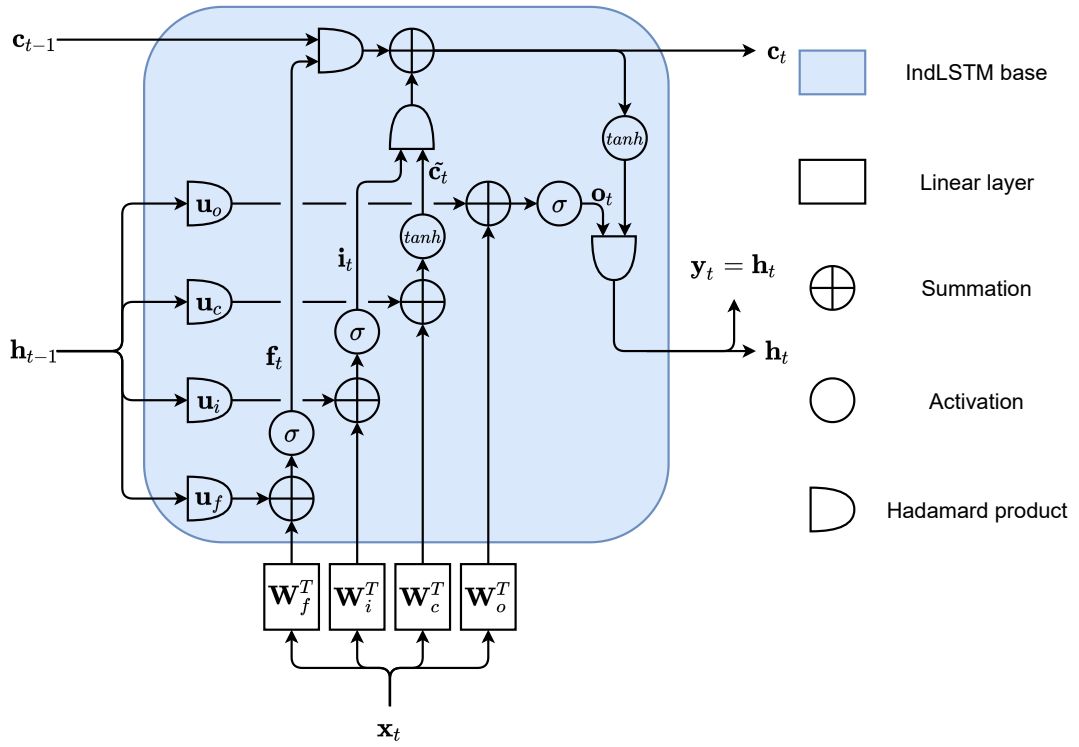


Figure 4.1: The structure of IndLSTM.

$$\mathbf{c}_t = \mathbf{f}_t \circ \mathbf{c}_{t-1} + \mathbf{i}_t \circ \tilde{\mathbf{c}}_t \quad (4.5)$$

$$\mathbf{y}_t = \mathbf{h}_t = \mathbf{o}_t \circ \tanh(\mathbf{c}_t) \quad (4.6)$$

Similar to the feedforward process of IndRNN in chapter 1, except for the linear weight matrices multiplied by the input vector $\mathbf{x}_t$ at each time step t , the rest of the computation only consists of summations and element-wise products, so the independency assumptions in IndLSTM are assured.

The feedback computations of IndLSTM are presented in Equation 4.7, Equation 4.8, Equation 4.9, Equation 4.10, Equation 4.11, Equation 4.12, Equation 4.13, Equation 4.14, Equation 4.15, Equation 4.16, Equation 4.17, Equation 4.18, Equation 4.19, Equation 4.20, Equation 4.21 and Equation 4.22.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{o}_t} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right) \circ \tanh(\mathbf{c}_t) \quad (4.7)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} + \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \right) \circ \mathbf{o}_t \circ \tanh'(\mathbf{c}_t) \quad (4.8)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{f}_t} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} \circ \mathbf{c}_{t-1} \quad (4.9)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{i}_t} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} \circ \tilde{\mathbf{c}}_t \quad (4.10)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t \circ \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} \quad (4.11)$$

$$\frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{c}}_t} = \mathbf{i}_t \circ \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} \quad (4.12)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_c^T} = \sum_t \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{c}}_t} \circ \tanh'(\mathbf{W}_c^T \mathbf{x}_t + \mathbf{u}_c \circ \mathbf{h}_{t-1}) \cdot \mathbf{x}_t^T \quad (4.13)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_c} = \sum_t \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{c}}_t} \circ \tanh'(\mathbf{W}_c^T \mathbf{x}_t + \mathbf{u}_c \circ \mathbf{h}_{t-1}) \circ \mathbf{h}_{t-1} \quad (4.14)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_o^T} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{o}_t} \circ \sigma'(\mathbf{W}_o^T \mathbf{x}_t + \mathbf{u}_o \circ \mathbf{h}_{t-1}) \cdot \mathbf{x}_t^T \quad (4.15)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_o} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{o}_t} \circ \sigma'(\mathbf{W}_o^T \mathbf{x}_t + \mathbf{u}_o \circ \mathbf{h}_{t-1}) \circ \mathbf{h}_{t-1} \quad (4.16)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_i^T} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{i}_t} \circ \sigma'(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{u}_i \circ \mathbf{h}_{t-1}) \cdot \mathbf{x}_t^T \quad (4.17)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_i} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{i}_t} \circ \sigma'(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{u}_i \circ \mathbf{h}_{t-1}) \circ \mathbf{h}_{t-1} \quad (4.18)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_f^T} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{f}_t} \circ \sigma'(\mathbf{W}_f^T \mathbf{x}_t + \mathbf{u}_f \circ \mathbf{h}_{t-1}) \cdot \mathbf{x}_t^T \quad (4.19)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_f} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{f}_t} \circ \sigma'(\mathbf{W}_f^T \mathbf{x}_t + \mathbf{u}_f \circ \mathbf{h}_{t-1}) \circ \mathbf{h}_{t-1} \quad (4.20)$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{x}_t} &= \mathbf{W}_c \cdot \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{c}}_t} \circ \tanh'(\mathbf{W}_c^T \mathbf{x}_t + \mathbf{u}_c \circ \mathbf{h}_{t-1}) \\ &\quad + \mathbf{W}_o \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{o}_t} \circ \sigma'(\mathbf{W}_o^T \mathbf{x}_t + \mathbf{u}_o \circ \mathbf{h}_{t-1}) \\ &\quad + \mathbf{W}_i \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{i}_t} \circ \sigma'(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{u}_i \circ \mathbf{h}_{t-1}) \\ &\quad + \mathbf{W}_f \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{f}_t} \circ \sigma'(\mathbf{W}_f^T \mathbf{x}_t + \mathbf{u}_f \circ \mathbf{h}_{t-1}) \end{aligned} \quad (4.21)$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-1}} &= \mathbf{u}_c \circ \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{c}}_t} \circ \tanh'(\mathbf{W}_c^T \mathbf{x}_t + \mathbf{u}_c \circ \mathbf{h}_{t-1}) \\ &\quad + \mathbf{u}_o \circ \frac{\partial \mathcal{L}}{\partial \mathbf{o}_t} \circ \sigma'(\mathbf{W}_o^T \mathbf{x}_t + \mathbf{u}_o \circ \mathbf{h}_{t-1}) \\ &\quad + \mathbf{u}_i \circ \frac{\partial \mathcal{L}}{\partial \mathbf{i}_t} \circ \sigma'(\mathbf{W}_i^T \mathbf{x}_t + \mathbf{u}_i \circ \mathbf{h}_{t-1}) \\ &\quad + \mathbf{u}_f \circ \frac{\partial \mathcal{L}}{\partial \mathbf{f}_t} \circ \sigma'(\mathbf{W}_f^T \mathbf{x}_t + \mathbf{u}_f \circ \mathbf{h}_{t-1}) \end{aligned} \quad (4.22)$$

Since the training of IndLSTM networks requires a lot of computation and storage, it is not favored in the experiments of this research compared to IndRevRNN. This issue is explained in chapter 5.

4.2 Basic IndRevRNN

Inspired by IndRNN and RevNet, the network structure of a basic IndRevRNN is shown in Figure 4.2. Simply put, it can be seen as a RevNet with n blocks nested in each time step of an IndRNN.

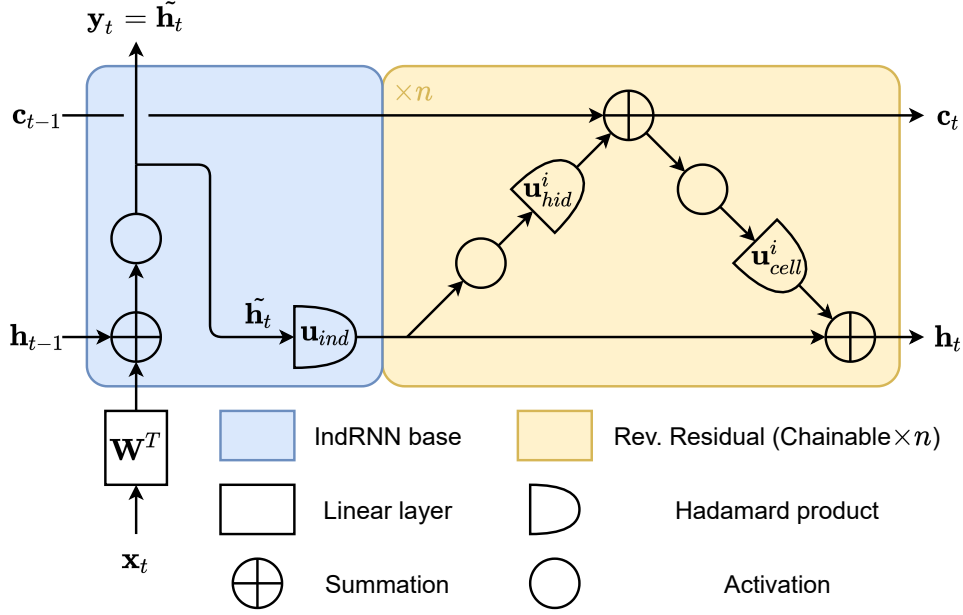


Figure 4.2: Basic IndRevRNN.

The blue part in Figure 4.2 is similar to an IndRNN in Figure 3.3. The only difference is that the weight vector $\mathbf{u}$ is multiplied by the output $\mathbf{y}_t = \tilde{\mathbf{h}}_t$ instead of the input $\mathbf{h}_{t-1}$ at time step t . This weight vector $\mathbf{u}$ is called an independently weight vector and denoted as $\mathbf{u}_{ind}$ in this thesis to distinguish it from other weight vectors. All time steps in the blue part share the same $\mathbf{u}_{ind}$.

The yellow part in Figure 4.2 is similar to an RevNet with n blocks in Figure 3.4, where n is an adjustable value. The blue part at each time step is followed by the same yellow part. Each block i in the yellow part has a hidden weight vector $\mathbf{u}_{hid}^i$ and a cell weight vector $\mathbf{u}_{cell}^i$ that are different from the ones in other blocks.

Since the total block number n of IndRevRNN will not be specified in advance in the theoretical derivation, the forward pass of the blue and the yellow parts in Figure 4.2 can only be listed separately, as shown in Equation 4.23, Equation 4.24, Equation 4.25, Equation 4.26, Equation 4.27, Equation 4.28 and Equation 4.29, where the lower right corner mark indicates the time step index t and the upper right corner mark indicates the block index i .

$$\mathbf{y}_t = \tilde{\mathbf{h}}_t = \sigma(\mathbf{W}^T \mathbf{x}_t + \mathbf{h}_{t-1}) \quad (4.23)$$

$$\mathbf{h}_t^0 = \mathbf{u}_{ind} \circ \tilde{\mathbf{h}}_t \quad (4.24)$$

$$\mathbf{c}_t^0 = \mathbf{c}_{t-1} \quad (4.25)$$

$$\mathbf{c}_t^i = \mathbf{c}_t^{i-1} + \mathbf{u}_{hid}^i \circ \sigma(\mathbf{h}_t^{i-1}) \quad (4.26)$$

$$\mathbf{h}_t^i = \mathbf{h}_t^{i-1} + \mathbf{u}_{cell}^i \circ \sigma(\mathbf{c}_t^i) \quad (4.27)$$

$$\mathbf{h}_t = \mathbf{h}_t^n \quad (4.28)$$

$$\mathbf{c}_t = \mathbf{c}_t^n \quad (4.29)$$

Note that the upper right corner mark once indicates the element index k in a vector when discussing the independence and parallel computing. Here the independency conditions still hold, i.e., each element k in vectors still independently participates in the computations. But for the sake of clarity, the equations use vector computations instead of independent scalar computations.

The corresponding backward pass of IndRevRNN is shown in Equation 4.30, Equation 4.31, Equation 4.32, Equation 4.33, Equation 4.34, Equation 4.35, Equation 4.36, Equation 4.37, Equation 4.38, Equation 4.39, Equation 4.40, Equation 4.41, Equation 4.42 and Equation 4.43.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^n} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t} \quad (4.30)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t^n} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \quad (4.31)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_{cell}^i} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t^i} \circ \sigma(\mathbf{c}_t^i) \quad (4.32)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^i} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t^i} \circ \mathbf{u}_{cell}^i \circ \sigma'(\mathbf{c}_t^i) \quad (4.33)$$

$$\mathbf{h}_t^{i-1} = \mathbf{h}_t^i - \mathbf{u}_{cell}^i \circ \sigma(\mathbf{c}_t^i) \quad (4.34)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_{hid}^i} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^i} \circ \sigma(\mathbf{h}_t^{i-1}) \quad (4.35)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t^{i-1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^i} \circ \mathbf{u}_{hid}^i \circ \sigma'(\mathbf{h}_t^{i-1}) \quad (4.36)$$

$$\mathbf{c}_t^{i-1} = \mathbf{c}_t^i - \mathbf{u}_{hid}^i \circ \sigma(\mathbf{h}_t^{i-1}) \quad (4.37)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_{t-1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \quad (4.38)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_{ind}} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t^0} \circ \tilde{\mathbf{h}}_t \quad (4.39)$$

$$\frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}_t} = \mathbf{u}_{ind} \circ \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t^0} \quad (4.40)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^T} = \sum_t \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} + \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}_t} \right) \circ \sigma'(\mathbf{W}^T \mathbf{x}_t + \mathbf{h}_{t-1}) \cdot \mathbf{x}_t^T \quad (4.41)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_t} = \mathbf{W} \cdot \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} + \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}_t} \right) \cdot \sigma'(\mathbf{W}^T \mathbf{x}_t + \mathbf{h}_{t-1}) \quad (4.42)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t-1}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}_t} + \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{h}}_t} \right) \circ \sigma'(\mathbf{W}^T \mathbf{x}_t + \mathbf{h}_{t-1}) \quad (4.43)$$

In the experiments, the matrix products of linear layers in Equation 4.23, Equation 4.41 and Equation 4.42 are completed outside the RNN layers in order to maintain independencies in computations.

There are two simple variations of the basic IndRevRNN in Figure 4.2, as shown in Figure 4.3 and Figure 4.4.

The difference between Figure 4.2 and Figure 4.3 is whether the independently weight vector $\mathbf{u}_{ind}$ can learn from the initial hidden state vector $\mathbf{h}_0$. This matter is related to work such as pretraining of recurrent networks [21] and stateful LSTM [33]. In some tests on the model, it is found that the structures in Figure 4.2 and Figure 4.3 have similar results, but the latter requires more space to store intermediate variables than the former, so the settings in Figure 4.2 are more favorable.

The difference between Figure 4.2 and Figure 4.4 is whether activation functions are placed before or after hidden weight vectors $\mathbf{u}_{hid}$ and cell weight vectors $\mathbf{u}_{cell}$ in the reversible residual blocks. This issue of activation function location is mentioned in [36], which compares the effect of pre-activations and post-activations. In some attempts, the latter model is less stable when using ReLU as activation function during training, and does not have a better result when using tanh and Sigmoid compared with the former, so this thesis applies the block in Figure 4.2.

The experiments and results of the basic IndRevRNN will be discussed in chapter 5.

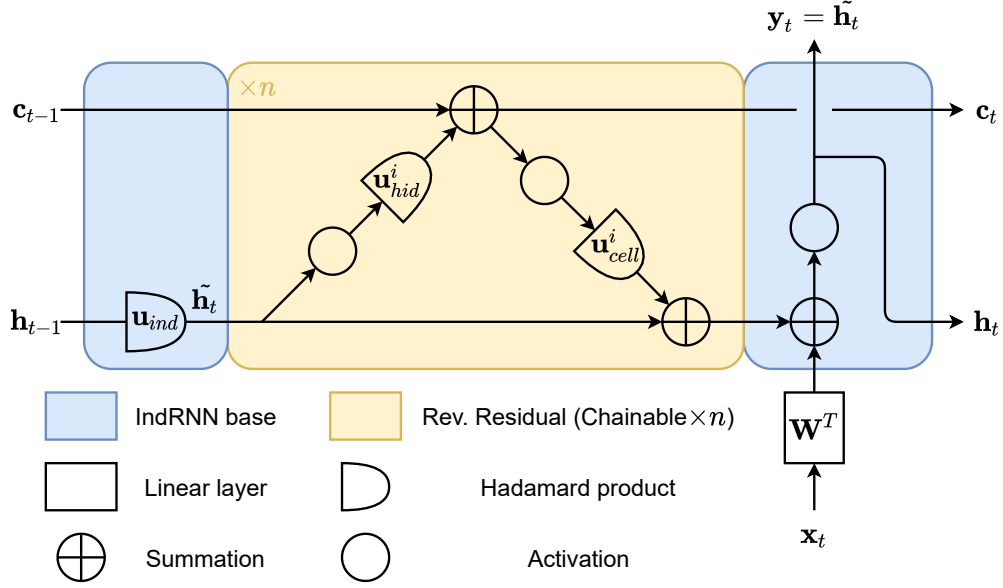


Figure 4.3: IndRevRNN with independently weight vector before reversible residual blocks.

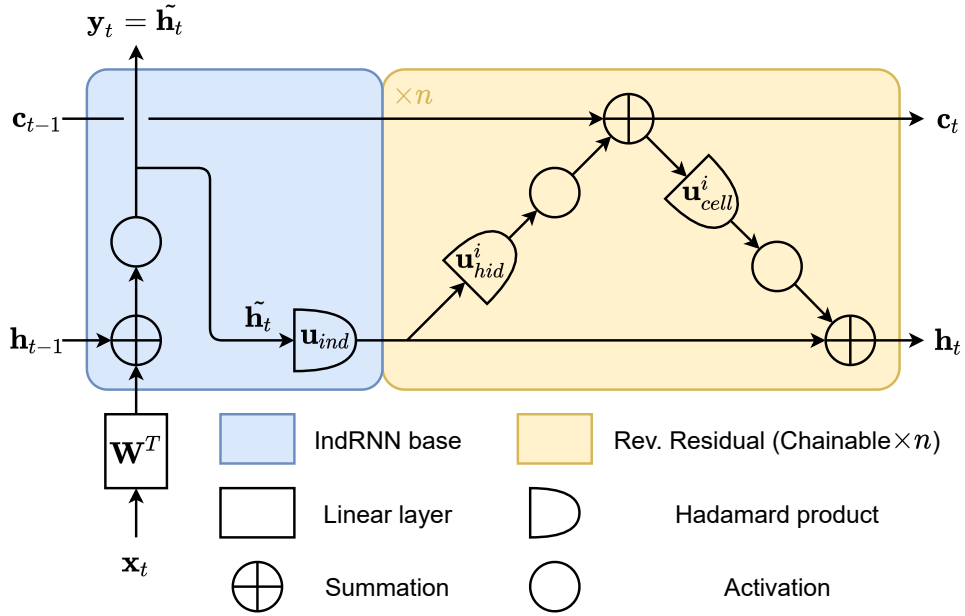


Figure 4.4: IndRevRNN with post residual activation functions.

In addition to the above two variations, there are other modifications to the basic IndRevRNN that have the potential to improve performance. These variants will be introduced in the next two sections.

4.3 Using cell factors to regulate long-term memory

Work such as [39] and [77] prove that it is beneficial to add a module that simulates long-term memories in RNNs, which is implemented by cell state vectors in the network of this thesis. However, this does not mean the cell state vector at each time step or in each reversible residual block should have the same significance, i.e., the new parameter introduced in Figure 4.5 can describe the change in the importance of cell state vector in each reversible residual block.

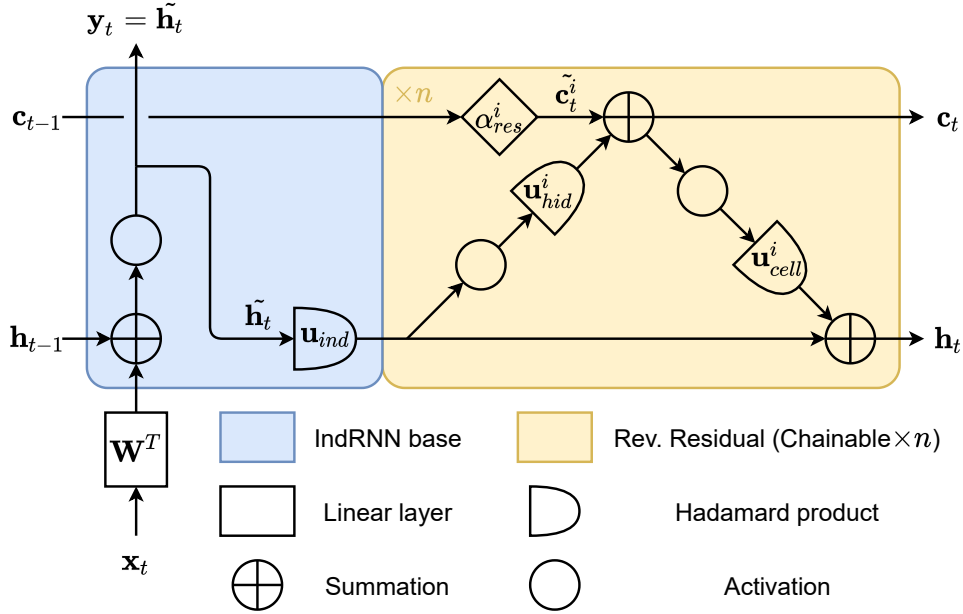


Figure 4.5: IndRevRNN with residual cell factors.

This new parameter is called a residual cell factor in Figure 4.5, denoted as α_{res}^i for the i -th reversible residual block. It is a scalar that differs in each block. When α_{res}^i is set to 1, the network structure is the same as that in Figure 4.2. When α_{res}^i is set to 0, all connections with cell state vectors will be severed, and the network will not generate or learn from long-term memory. When α_{res}^i is set to a value between 0 and 1, the network will merge a regulated version of cell state vector $\tilde{\mathbf{c}}_t^i$ into the hidden state vector after the Hadamard product in each block. That is, the summation of long-term memory with

regulation and short-term memory after processing leads to the new long-term memory. Since the sign of each element in a cell state vector is alterable, the sign of α_{res}^i does not affect the output.

In the experiments, α_{res}^i is set to a learnable value. The forward pass and backward pass of this computation are shown in Equation 4.44, Equation 4.45 and Equation 4.46.

$$\tilde{\mathbf{c}}_t^i = \alpha_{res}^i \cdot \mathbf{c}_t^{i-1} \quad (4.44)$$

$$\frac{\partial \mathcal{L}}{\partial \alpha_{res}^i} = \sum_t \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{c}}_t^i} \cdot \mathbf{c}_t^{i-1} \quad (4.45)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^{i-1}} = \alpha_{res}^i \cdot \frac{\partial \mathcal{L}}{\partial \tilde{\mathbf{c}}_t^i} \quad (4.46)$$

The parameter cell factor can not only be used in each reversible residual block i but also be placed at each time step t , as demonstrated in Figure 4.6. In this case, this parameter is called a sequence cell factor, denoted as α_{seq}^t .

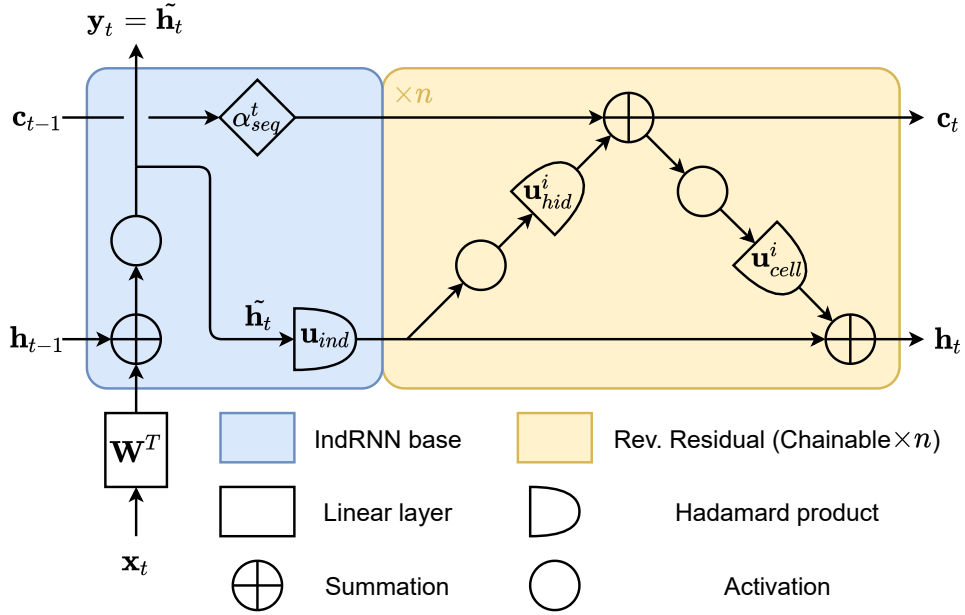


Figure 4.6: IndRevRNN with sequence cell factors.

Similar to the residual cell factor α_{res}^i , when α_{seq}^t is set to 1, the network structure is the same as that in Figure 4.2. When α_{seq}^t is set to 0, $\mathbf{c}_t^0$ at the beginning of reversible residual blocks will be a zero vector, i.e., the long-term memory will only be maintained in

the reversible residual blocks of the same time step. When α_{seq}^t is set to a value between 0 and 1, it also plays a role in regulating long-term memory. Similarly, the sign of α_{seq}^t does not affect the output.

The computation process for setting α_{seq}^t as a learnable parameter is shown in Equation 4.47, Equation 4.48 and Equation 4.49.

$$\mathbf{c}_t^0 = \alpha_{seq}^t \cdot \mathbf{c}_{t-1} \quad (4.47)$$

$$\frac{\partial \mathcal{L}}{\partial \alpha_{seq}^t} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \cdot \mathbf{c}_{t-1} \quad (4.48)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_{t-1}} = \alpha_{seq}^t \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \quad (4.49)$$

Some deformed variants of cell factors are shown in Figure 4.7. The experiments and results of applying cell factors will be discussed in chapter 5.

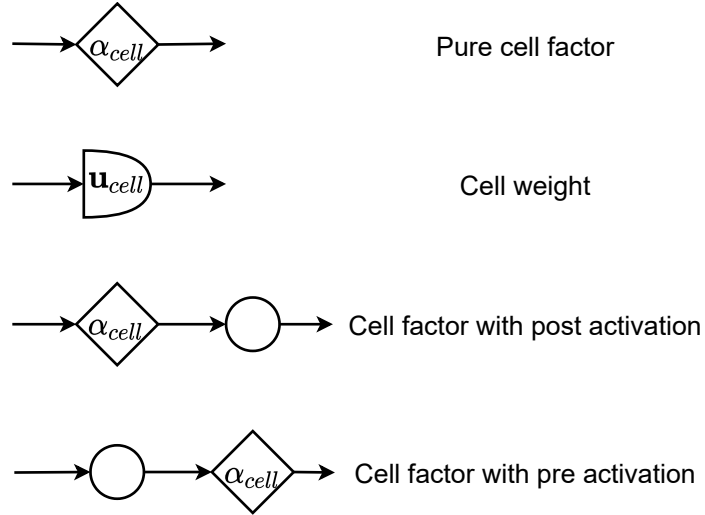


Figure 4.7: Deformed variants of cell factors.

4.4 Using forget gates as trainable information control units

Work such as [83] and [88] have analyzed the effectiveness of using forget gates in LSTM networks to control the information flow. This proposal can also be applied to IndRevRNN. Figure 4.8, Figure 4.9 and Figure 4.10 show some IndRevRNN variants with forget gates.

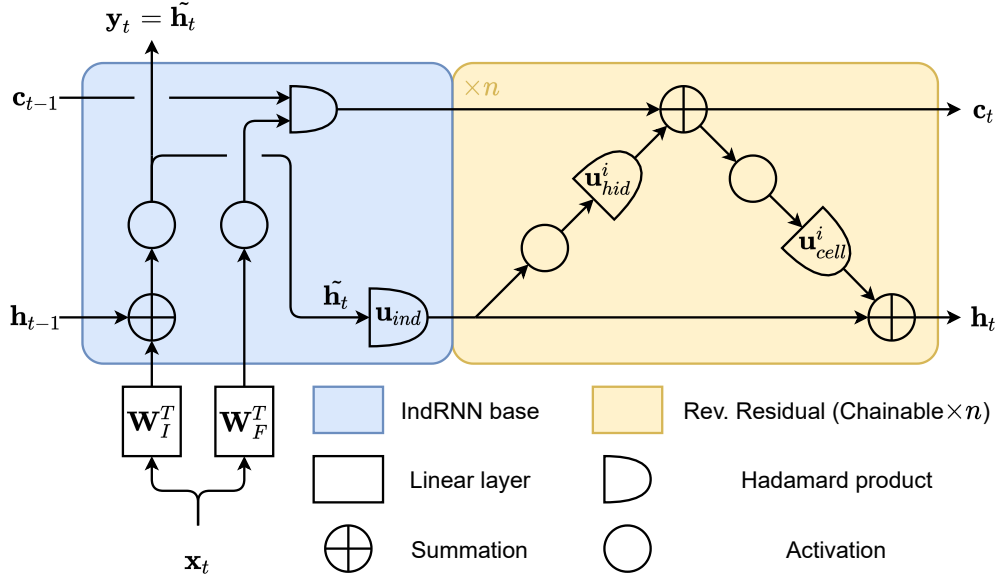


Figure 4.8: IndRevRNN with forget gate from input.

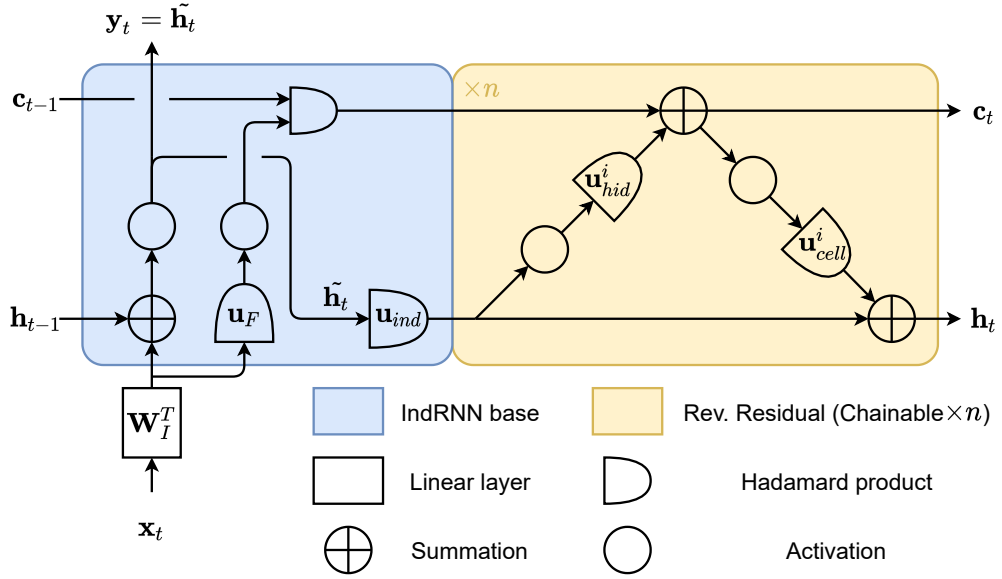


Figure 4.9: IndRevRNN with forget gate from before recurrent.

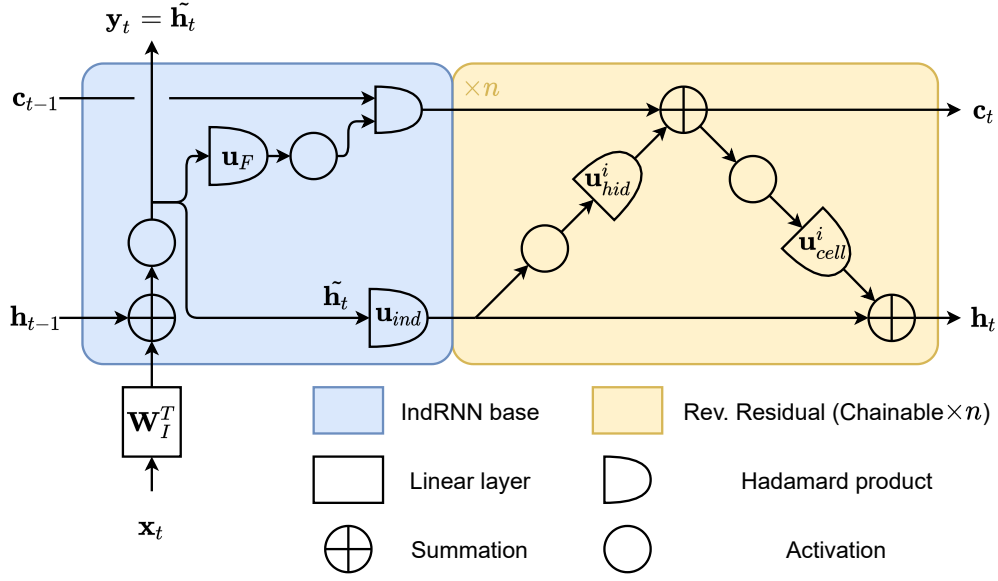


Figure 4.10: IndRevRNN with forget gate from after activation.

In Figure 4.8, the forget gate is directly derived from the input vector $\mathbf{x}_t$ at each time step t , so this type of forget gate is called the forget gate from input. $\mathbf{x}_t$ is respectively multiplied by the input weight matrix $\mathbf{W}_I^T$ and the forget weight matrix $\mathbf{W}_F^T$ in the linear layer. The result of the former multiplication participates in the computation similar to that in Figure 4.2, and the result of the latter multiplication is brought into an activation function to form the forget gate, which then does an element-wise product with the cell state vector $\mathbf{c}_{t-1}$. The forward pass and backward pass of this process are listed in Equation 4.50, Equation 4.51, Equation 4.52 and Equation 4.53.

$$\mathbf{c}_t^0 = \sigma(\mathbf{W}_F^T \mathbf{x}_t) \odot \mathbf{c}_{t-1} \quad (4.50)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_F^T} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \odot \sigma'(\mathbf{W}_F^T \mathbf{x}_t) \odot \mathbf{c}_{t-1} \cdot \mathbf{x}_t^T \quad (4.51)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_t} = \mathbf{W}_F \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \odot \sigma'(\mathbf{W}_F^T \mathbf{x}_t) \odot \mathbf{c}_{t-1} \quad (4.52)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_{t-1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \odot \sigma(\mathbf{W}_F^T \mathbf{x}_t) \quad (4.53)$$

In Figure 4.9, the forget gate is added after the linear layer and before the recurrent layer, so this type of forget gate is called the forget gate from before recurrent. At time

step t , the input vector $\mathbf{x}_t$ after the linear layer $\mathbf{W}_I^T$ does an element-wise product with a forget weight vector $\mathbf{u}_F$, and the result is brought into an activation function to form the forget gate for the cell state vector $\mathbf{c}_{t-1}$. The forward pass and backward pass of this process are listed in Equation 4.54, Equation 4.55 and Equation 4.56.

$$\mathbf{c}_t^0 = \sigma(\mathbf{W}_I^T \mathbf{x}_t \circ \mathbf{u}_F) \circ \mathbf{c}_{t-1} \quad (4.54)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_F} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \circ \mathbf{W}_I^T \mathbf{x}_t \circ \sigma'(\mathbf{W}_I^T \mathbf{x}_t \circ \mathbf{u}_F) \circ \mathbf{c}_{t-1} \quad (4.55)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_{t-1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \circ \sigma(\mathbf{W}_I^T \mathbf{x}_t \circ \mathbf{u}_F) \quad (4.56)$$

In Figure 4.10, the forget gate is added after the activation function of the recurrent layer, so this type of forget gate is called the forget gate from after activation. In this case, the output vector $\mathbf{y}_t = \tilde{\mathbf{h}}_t$ at each time step t is used as input of the forget gate. This vector undergoes an element-wise product with the cell state vector $\mathbf{c}_{t-1}$ after the computation with a forget weight vector $\mathbf{u}_F$ and in the activation function. The forward pass and backward pass of this process are listed in Equation 4.57, Equation 4.58 and Equation 4.59.

$$\mathbf{c}_t^0 = \sigma(\mathbf{u}_F \circ \tilde{\mathbf{h}}_t) \circ \mathbf{c}_{t-1} \quad (4.57)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_F} = \sum_t \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \circ \tilde{\mathbf{h}}_t \circ \sigma'(\mathbf{u}_F \circ \tilde{\mathbf{h}}_t) \circ \mathbf{c}_{t-1} \quad (4.58)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{c}_{t-1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t^0} \circ \sigma(\mathbf{u}_F \circ \tilde{\mathbf{h}}_t) \quad (4.59)$$

The experiments and results of applying forget gates on models will be discussed in chapter 5.

5 Experiments and results

This chapter will first analyze how to initialize the model parameters in IndRevRNN. Next, the performance of IndRevRNN and its variants on the permuted sequential MNIST dataset and Penn Treebank corpus will be discussed.

5.1 Parameter initialization

Work such as [28], [35] and [50] prove that proper parameter initialization plays a vital role in the convergence and stability of network training. Therefore, this thesis has done some research on parameter initialization of IndRevRNN.

As shown in Figure 5.1, the IndRevRNN has three groups of parameters that can be initialized and they will be analyzed separately.

The first group of parameters includes the independently weight vector $\mathbf{u}_{ind}$ shared by each time step, as well as the hidden weight vector $\mathbf{u}_{hid}^i$ and the cell weight vector $\mathbf{u}_{cell}^i$ shared by each reversible residual block.

As described in [57] and [56], in order to avoid the exploding and vanishing gradient problems, each element in $\mathbf{u}_{ind}$ is initialized with a random value between 0 and 1, i.e., $\mathbf{u}_{ind} \sim \mathcal{U}(0, 1)$. In order to tackle longer sequences, the upper bound of this uniform distribution is further set to 0.99 in some occasions, that is $\mathbf{u}_{ind} \sim \mathcal{U}(0, 0.99)$.

$\mathbf{u}_{hid}^i$ and $\mathbf{u}_{cell}^i$ can be initialized in the same way as $\mathbf{u}_{ind}$. But a better way is to assume that the two paths connecting the cell state vector and the hidden state vector in each reversible residual block are originally disconnected, so that the network won't learn invalid information or noise from long-term memory at the beginning.

However, all weights in $\mathbf{u}_{hid}^i$ and $\mathbf{u}_{cell}^i$ can't be initialized to $\mathbf{0}$, otherwise no information can go from the cell state to the hidden state or vice versa in the forward pass, and the elements in weight vectors won't be updated in the backward. Therefore, this thesis initializes the elements in $\mathbf{u}_{hid}^i$ and $\mathbf{u}_{cell}^i$ to have a 0 mean and conform to the Gaussian distribution with a standard deviation of 10^{-4} , namely $\mathbf{u}_{hid}^i \sim \mathcal{N}(0, 10^{-8})$ and $\mathbf{u}_{cell}^i \sim \mathcal{N}(0, 10^{-8})$. In this case, the network will gradually update each weight element that participates in the connection between long-term memory and short-term memory during training.

The second group of parameters includes the initial hidden state vector $\mathbf{h}_0$ and the initial cell state vector $\mathbf{c}_0$. In many studies on LSTM, such as [80] and [51], these two

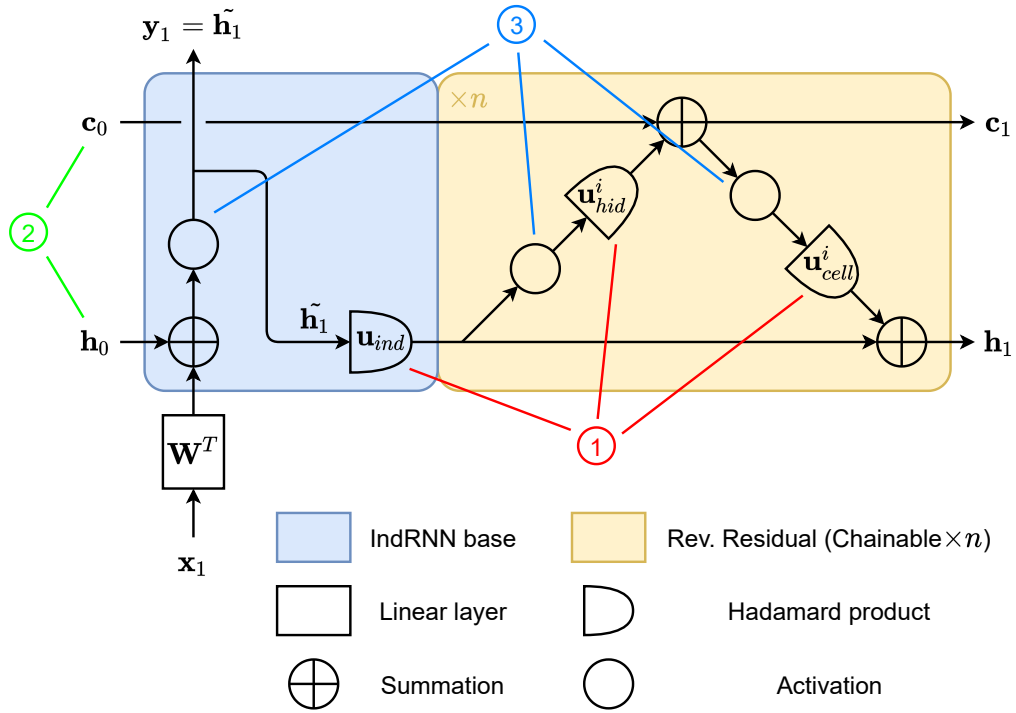


Figure 5.1: Three groups of parameters to initialize in the basic IndRevRNN.

vectors are initialized to $\mathbf{0}$, which is also adopted in this thesis.

But in the case of cutting a long sequence into multiple short sequences, the $\mathbf{h}_n$ and $\mathbf{c}_n$ at the end of the previous short sequence can also be used as the $\mathbf{h}_0$ and $\mathbf{c}_0$ at the beginning of the next short sequence. This scenario is usually applied in the training of natural language processing models.

Besides, when applying ReLU as residual activation function, initializing $\mathbf{c}_0$ to a zero vector will make some vector elements become infinite during training, which is caused by the ReLU function being discontinuous at 0. This problem can be solved by making $\mathbf{c}_0$ conform to a normal distribution with a 0 mean and a standard deviation of 10^{-2} , namely $\mathbf{c}_0 \sim \mathcal{N}(0, 10^{-4})$. The initialization of $\mathbf{h}_0$ has no effect on this problem.

The third group of parameters includes the types of three activation functions in the blue part and yellow part. Work such as [64] and [67] have compared the influence of different activation functions on the weight update.

In this thesis, the activation function in the blue part or the independently activation function, is consistent with the one in [56] to take full advantage of independency assumptions. There are three options for activation functions in the yellow part or the residual activation functions, ReLU, tanh and Sigmoid. Experiments show that tanh and Sigmoid bring similar results. To maintain robustness, there are not many attempts that use ReLU as residual activation functions.

The selection of activation functions needs further exploration.

Table 5.1 summarizes how all model parameters are initialized.

Table 5.1: Initialization of model parameters.

Model parameters	Initialization
Independently weight vector $\mathbf{u}_{ind}$	$\mathcal{U}(0, 1)$ or $\mathcal{U}(0, 0.99)$
Hidden weight vector $\mathbf{u}_{hid}^i$	$\mathcal{N}(0, 10^{-8})$
Cell weight vector $\mathbf{u}_{cell}^i$	$\mathcal{N}(0, 10^{-8})$
Initial hidden state vector $\mathbf{h}_0$	$\mathbf{0}$ or $\mathbf{h}_n$ of the last sequence
Initial cell state vector $\mathbf{c}_0$	$\mathbf{0}$ or $\mathcal{N}(0, 10^{-4})$ or $\mathbf{c}_n$ of the last sequence
Independently activation function	ReLU
Residual activation functions	tanh or Sigmoid

5.2 Experiment results on the MNIST dataset

The MNIST dataset, short for the Modified National Institute of Standards and Technology dataset, is a database of handwritten digits that is widely used in the field of image

processing and machine learning [53]. The MNIST dataset contains 60,000 training images and 10,000 test images, each of which is a grayscale image of a digit labeled from 0 to 9 [52]. Some example images in the MNIST dataset are shown in Figure 5.2.

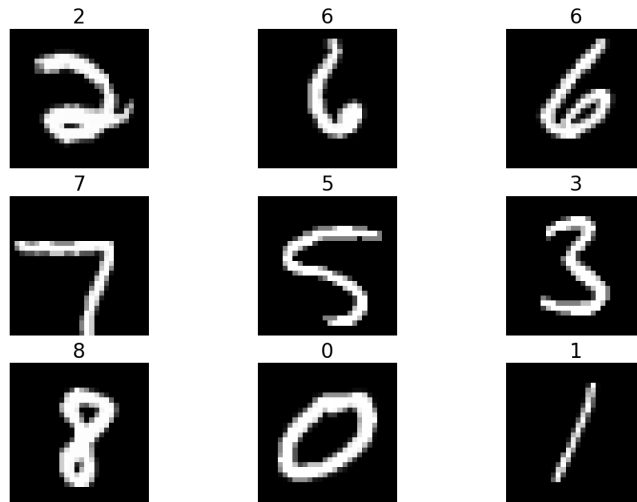


Figure 5.2: Images in the MNIST dataset.

Images in Figure 5.2 have a total of 784 pixels and they are often used to train classification models. In the classification task with recurrent neural networks, the images are often compressed into a sequence of 784 time steps. This leads to the sequential MNIST dataset, where two adjacent time steps retain the spatial correlations as in the original image.

To train RNNs that better characterize the relevance between two time steps that are far away from each other, one usually shuffles the pixels in an MNIST image with random permutation after compressing them, and this creates the permuted sequential MNIST dataset [30]. Figure 5.3 shows some example images of the permuted sequential MNIST dataset in a two-dimensional space.

In general, the prediction accuracy of RNNs on the sequential MNIST dataset will be higher than that on the permuted sequential MNIST dataset. To ensure that different RNNs can be compared under a unified standard, Figure 5.4 displays the network model used in the MNIST classification, which is not the optimal model for best results.

90% of the images from the training set are used for training, while the remaining 10% are used for validation.

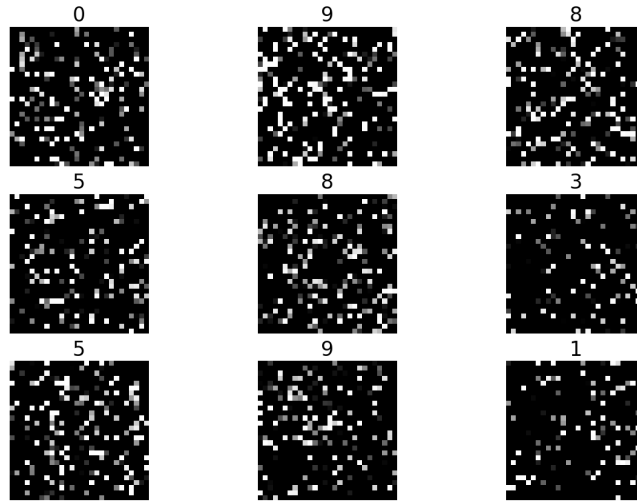


Figure 5.3: Images in the permuted sequential MNIST dataset in a 2D space.

Limited by the graphics memory, the batch size of the model is 32, i.e., the input of the network is a mini-batch containing 32 sequences, and each sequence contains 784 pixels. To ease data processing, the input is viewed to a tensor with shape $784 \times 32 \times 1$. The first linear layer expands the shape of the input tensor to $784 \times 32 \times 128$, making it correspond to the RNN layer that contains 128 hidden neurons.

In the RNN layer that uses IndLSTM or IndRevRNN, the initial hidden state vector $\mathbf{h}_0$ and cell state vector $\mathbf{c}_0$ both have shape 32×128 . The independently weight vector $\mathbf{u}_{ind}$ or weight vectors in IndLSTM have shape 1×128 , while the hidden weight vector $\mathbf{u}_{hid}$ and cell weight vector $\mathbf{u}_{cell}$ have shape $n \times 128$ depending on the reversible residual block length n .

The IndRevRNN or IndLSTM layer is followed by a Batch Normalization layer which normalizes the time step dimension and batch size dimension. For uniform comparison, here the Batch Normalization layer uses the original Batch Normalization strategy compared to the Recurrent Batch Normalization strategy [19].

The Batch Normalization layer is followed by a Dropout layer. For uniform comparison, here the Dropout layer uses the per step dropout pattern compared to the Variational Dropout pattern [26].

The above-mentioned architecture **Linear layer - IndRevRNN or IndLSTM layer - Batch Normalization layer - Dropout layer** is repeated 6 times to build a deep

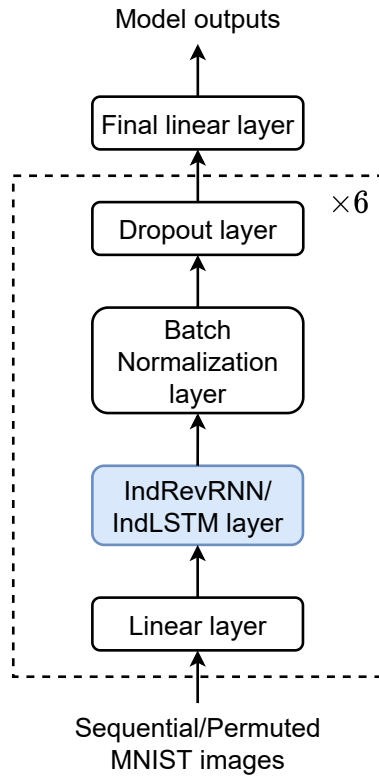


Figure 5.4: Model architecture used for experiments on the permuted sequential MNIST dataset.

network for learning high-dimensional features. The shape of the tensor after repetitions is still $784 \times 32 \times 128$.

One last linear layer will be used on the last time step of this tensor to make it correspond to the 10 classes of the MNIST dataset, changing the shape from 32×128 to 32×10 . The cross-entropy loss of the output tensor and the class labels of 32 sequences is then obtained.

Networks are trained using the Adam optimizer with an initial learning rate of 2×10^{-4} [46]. The learning rate is halved if the metric has no improvement in 20 epochs.

Under the above configuration, when selecting IndLSTM for RNN layers, the test accuracy of the model on the permuted sequential MNIST dataset is 94.27%. This is an acceptable result, but the training on the NVIDIA GeForce Titan X with 12 GB VRAM takes an average of 150 seconds per epoch, while it takes an average of only 54 seconds per epoch when selecting IndRevRNN without reversible residual blocks. As the IndLSTM network needs massive computation and storage, it is not favored compared to the IndRevRNN.

A comparison of training time per epoch of independently recurrent networks with different architectures under the above configuration on the permuted sequential MNIST dataset is shown in Table 5.2.

Table 5.2: Average training time per epoch of different independently recurrent networks on the permuted sequential MNIST dataset.

Independently recurrent networks	Average training time per epoch in seconds
IndLSTM	150
IndRevRNN with no block	54
IndRevRNN with 1 block	64
IndRevRNN with 2 blocks	72
IndRevRNN with 3 blocks	81
IndRevRNN with 4 blocks	89

When choosing IndRevRNN for RNN layers, depending on the network variant and how many reversible residual blocks are used, the average test accuracies of the model are shown in Table 5.3. Among them, the accuracy of 94.42% is the average value obtained using all different network variants when the reversible residual block length is 0.

As can be seen from Table 5.3, using reversible residual blocks with different lengths in the basic IndRevRNN can slightly improve the classification accuracy on the MNIST dataset, i.e., the performance of the basic IndRevRNN in this task is slightly better than that of the IndRNN under the above configuration.

Other IndRevRNN variants further improve the test accuracy by about 0.5%. However,

Table 5.3: Average test accuracies of different IndRevRNN variants on the permuted sequential MNIST dataset.

Average test accuracies IndRevRNN variant	Block length				
	0	1	2	3	4
Basic	94.42%	94.56%	94.74%	94.68%	95.00%
With residual cell factors	94.42%	95.00%	95.46%	95.24%	95.63%
With sequence cell factors	94.42%	97.31%	97.47%	97.45%	97.50%
With forget gate from input	94.42%	95.35%	95.19%	95.35%	95.58%
With forget gate from before recurrent	94.42%	95.17%	95.71%	95.35%	95.53%
With forget gate from after activation	94.42%	95.00%	95.58%	94.88%	95.56%

there is no obvious correspondence between the test accuracy and the reversible residual block length. The difference in accuracies may only come from the randomness in the initialization and training process of experiments. More research is needed on this issue.

One good thing is that choosing a larger block length won't cause overfitting, although it increases computation and storage. This ensures that the network can always give stable results when a larger block length is chosen if complicated feature correlations between adjacent time steps need to be learned.

The variant that has the best performance under the above configuration is the IndRevRNN with sequence cell factors, with the best test accuracy of 97.50% on the permuted sequential MNIST dataset. But the problem of this variant is that the sequence length needs to be specified in advance, like 784 in this case. Choosing cell factor with post-activation in Figure 4.7 makes the training of IndRevRNN more stable.

Table 5.4 compares the best performance of IndRevRNN under the above configuration and some state-of-the-art RNNs on the permuted sequential MNIST dataset. In the experiments, using simple Dropout and simple Batch Normalization limits the performance of IndRevRNN. The IndRevRNN may perform better if other methods such as [26] and [19] are used.

5.3 Experiment results on the Penn Treebank corpus

The Penn Treebank or PTB is a broadly used corpus for evaluation of sequence learning models [60]. It is usually divided into three subsets for training, validation and test, containing approximately 900,000, 100,000 and 100,000 tokens, respectively [23]. Some example sentences in the PTB corpus are shown in Figure 5.5.

Table 5.4: Test accuracies of IndRevRNN and state-of-the-art RNNs on the permuted sequential MNIST dataset.

Types of RNNs	Test accuracies
LSTM [7]	88%
BN LSTM [19]	95.4%
IndRNN [57]	96%
LMU [85]	97.2%
Dense IndRNN [56]	97.2%
coRNN [70]	97.34%
Basic IndRevRNN	95%
IndRevRNN with forget gates	95.7%
IndRevRNN with sequence cell factors	97.5%

the bush white house is breeding <unk> ducks the s
ame way the nixon white house did it <unk> on an i
ssue that is <unk> cleaner air better treatment of
the disabled better child care

we would have thought by now though that there was
a significant core of people involved in governme
nt life who understood that direct taxation is n't
the only way to slow down an economy

foreign exchange dealers said that the currency ma
rket has begun to distance itself from the volatil
e stock exchange which has <unk> the market since
oct. N when the dow jones industrial average plung
ed more than N points

Figure 5.5: Example sentences in PTB.

Perplexity is a way of evaluating language models in natural language processing, which are probability distributions of a word or sentence given its previous text [44]. For evaluating RNN models trained on the PTB corpus, perplexity is chosen as the metric in this thesis.

Figure 5.6 displays the network model used on the PTB corpus. The architecture is shared by all the IndRevRNN variants for a unified comparison.

The texts in the training, validation and test sets of PTB corpus are cut into sequences of length 50. Limited by the graphics memory, the batch size of the model is 128. So the input in the training process is viewed to a tensor with shape 50×128 . This input is firstly processed by an encoder consisting of an embedding function with dropout as described in [63], and then processed by a dropout layer for word embeddings as described in [26].

Following the above two layers are three sets of alternating dense layers and transition layers. The transition layer in each of the three groups is preceded by 8, 6 and 4 dense layers respectively.

In each dense layer, the input goes through two sets of **Linear layer - Batch Normalization layer - IndRevRNN or IndLSTM layer - Dropout layer** structure, and the result will be concatenated with the layer input. The shape of the parameters in IndRevRNN or IndLSTM layer is determined by the output of the previous Linear layer. The Batch Normalization layer uses the Recurrent Batch Normalization as proposed in [19], and the Dropout layer uses the per sequence dropout mask on the input as proposed in [26].

In each transition layer, the input goes through one set of **Linear layer - Batch Normalization layer - IndRevRNN or IndLSTM layer - Dropout layer** structure. The settings of this structure are similar to that of a dense layer.

A final linear layer works as the decoder of the network. The shape of the final output is 1600×10000 .

Similar to the experiments on the MNIST dataset, networks are trained using the Adam optimizer with an initial learning rate of 2×10^{-4} [46]. The learning rate is halved if the metric has no improvement in 100 epochs. The IndLSTM network is also deprecated after some attempts due to massive computation and storage.

A comparison of training time per epoch of IndRevRNNs with different block lengths under the above configuration on the PTB corpus is shown in Table 5.5.

Under the above configuration, when selecting IndRevRNN for RNN layers, determined by the network alternative and the reversible residual block length, the average test perplexities of the model are shown in Table 5.6. Here the IndRevRNN variant with sequence cell factors is not used since the sequences in the PTB corpus don't need to have a fixed length like those in the permuted sequential MNIST dataset. In the results, the perplexity of 56.40 is the average value obtained using all different network variants

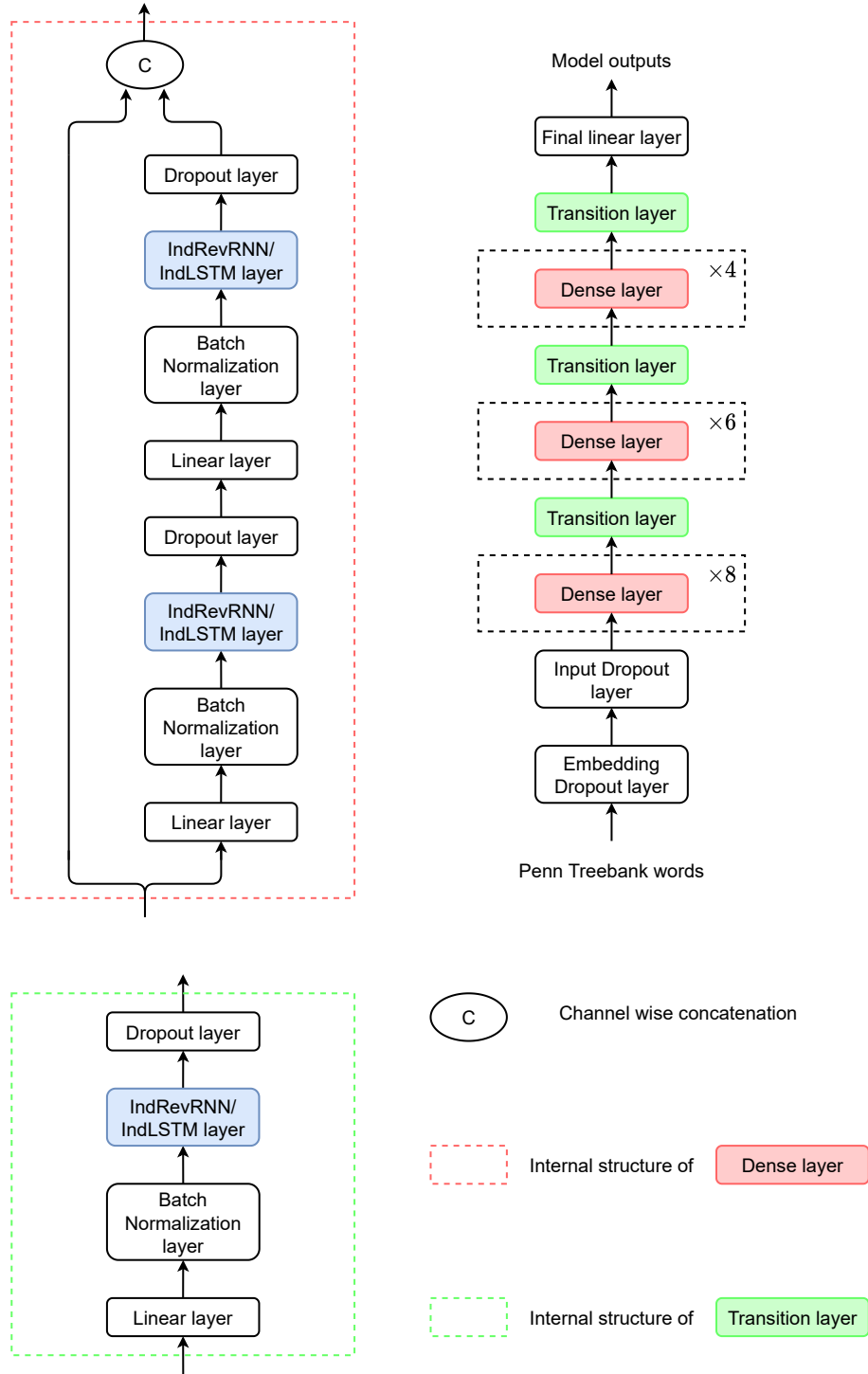


Figure 5.6: Model architecture used for experiments on the PTB corpus.

Table 5.5: Average training time per epoch of IndRevRNNs with different block lengths on the PTB corpus.

IndRevRNNs	Average training time per epoch in seconds
IndRevRNN with no block	71
IndRevRNN with 1 block	80
IndRevRNN with 2 blocks	78
IndRevRNN with 3 blocks	81
IndRevRNN with 4 blocks	85

when the reversible residual block length is 0.

Table 5.6: Average test perplexities of different IndRevRNN variants on the PTB corpus.

Test perplexities IndRevRNN variant	Block length				
	0	1	2	3	4
Basic	56.44	56.77	56.53	57.00	56.80
With residual cell factors	56.44	56.72	56.96	55.94	56.44
With forget gate from input	56.44	56.56	56.59	56.28	56.59
With forget gate from before recurrent	56.44	56.41	56.76	56.86	56.67
With forget gate from after activation	56.44	55.97	56.72	56.23	56.82

As can be seen from Table 5.6, the use of IndRevRNN variants does not improve the test perplexity on the PTB corpus compared to the IndRevRNN without reversible residual blocks, i.e., the original IndRNN. One possible reason for the result is that there are no complex feature correspondences between two adjacent time steps in a Penn Treebank sequence, namely the relationship between two neighboring words in a sentence of natural languages doesn't require too complex network models to learn.

Another possible reason is that sentences in natural languages tend to be very short. For example, an English sentence contains only 15 to 20 words on average, and normally no more than 40 [20]. Hence, there is no need to simulate long-term memories with cell state factors in short sequences containing up to 50 time steps.

From the above two reasons, it can be assumed that the IndRevRNN is more suitable for learning long sequences with complex dependencies.

Although the use of reversible residual blocks and other variant structures doesn't improve the performance of the network on the PTB corpus, it won't make the results

worse. So there is still potential in applying IndRevRNN to the learning of simple and short sequences as there is no risk of overfitting.

Table 5.7 compares the best performance of IndRevRNN under the above configuration and some state-of-the-art RNNs on the PTB corpus.

Table 5.7: Test perplexities of IndRevRNN and state-of-the-art RNNs on the PTB corpus.

Types of RNNs	Test perplexities
GRU [11]	92.48
LSTM [11]	78.93
Recurrent highway networks [92]	65.4
AWD-LSTM [61]	57.3
Dense IndRNN [56]	56.37
AWD-LSTM-MoS [89]	54.44
Trellis Network [12]	54.19
Basic IndRevRNN	56.7
IndRevRNN with forget gates	56.6
IndRevRNN with cell factors	56.4

6 Conclusion and future work

This chapter will review the main content of the whole thesis and propose some feasible future work.

6.1 Conclusion and discussions

This thesis proposes a new RNN structure, the IndRevRNN. The design of this structure is inspired by IndRNN and RevNet. It retains the advantages of previous networks, i.e., it solves the exploding and vanishing gradient problems and it performs parallel computing through CUDA, making the computation efficient and fast.

By introducing the reversible residual blocks into the network, the IndRevRNN has new advantages over IndRNN. It can better learn the correlations between two adjacent time steps of a long or scalable sequence, and more easily tackle the problem of long-term dependencies with the introduced cell state vectors.

Although the architecture becomes more complex, the properties of the reversible residual blocks allow the model to be trained without requiring massive storage or computation. The training time of the new model increases only slightly compared to that of the IndRNN model without blocks.

There are also some variants of the basic IndRevRNN, which differ slightly from the original network in the processing of long-term memory. One modification is applying cell factors to each time step or each reversible residual block that directly regulates cell state vectors. Another modification is using forget gates to control long-term memory with the information from input or intermediate variables.

Experiments show that the IndRevRNN and its variants with different block lengths have slightly better performance than the IndRNN in the classification task using permuted sequential MNIST dataset, while they have comparable performance in terms of perplexity to the IndRNN on Penn Treebank corpus. In particular, the IndRevRNN with sequence cell factors is especially suitable for the learning of fixed-length sequences, although it does not apply to non-fixed-length ones.

To sum up, the IndRevRNN proposed in this thesis is a novel RNN that exhibits state-of-the-art performance. It has high efficiency and can easily be applied to various sequence learning scenarios.

6.2 Future work

In addition to what has been done in this thesis, there is some future work related to the network design and experiments.

The model structure has some other possibilities. One can apply other dropout methods, such as Zoneout from [49], Gaussian Dropout from [86] and Variational Dropout from [47] to the hidden states and cell states, which may be better than simply applying a dropout mask to the input of the recurrent layer.

Designing models that combine neural networks and attention mechanisms is a hot research topic in deep learning. The ideas of [10] and [84] can also be used in IndRevRNN to improve the performance of the model.

Commonly used approaches in designing language learning networks also include dynamic evaluation [48], Mixture of Softmaxes or MoS [89] and ASGD Weight-Dropped or AWD [61], etc. All these ideas can be introduced into the network of this thesis.

Since there are several network variants proposed in this paper, experiments have only been performed on two datasets, namely the permuted sequential MNIST dataset and Penn Treebank corpus. Some other datasets are very suitable for examining the performance of RNNs, such as Human Activity Recognition [6] and NTU RGB+D [76].

In addition, the IndRevRNN can be applied to other scalable sequences, such as stock price sequences, video sequences, climate change sequences, music sequences, etc. Applying IndRevRNN to longer sequences with more complex correlations may lead to good results.

A Implementation of the IndRevRNN algorithm

This chapter will show the implementation of the basic IndRevRNN. The first section explains the pure Python program of IndRevRNN and some other corresponding functions. The second section explains the program written using custom CUDA extensions provided by Pytorch, which is suitable for parallel computing in IndRevRNN with independency assumptions. The third section shows the class module that calls the IndRevRNN functions and the initialization of model parameters.

A.1 Python baseline of IndRevRNN

Before all work starts, the required libraries are imported. These include Optional and Tuple for extended data types, Enum and unique for variable binding, nn and Tensor for building neural network modules and tensor computation, and Function for defining custom forward and backward functions.

```
from typing import Optional, Tuple
from enum import Enum, unique
import torch
from torch import nn, Tensor
from torch.autograd import Function
```

The Python class that wraps the IndRevRNN algorithm is defined below.

```
class _IndRevRNNTFuncPy(Function):
    @staticmethod
    def forward(ctx, y: Tensor, h_0: Tensor, c_0: Tensor, ind_weights:
        ↪ Tensor, hidden_weights: Tensor, cell_weights: Tensor, res_len:
        ↪ int, ind_act_typ: int, res_act_typ: int) -> Tensor:
        out, all_h, all_c = indrevrnn_py_fwd(y, h_0, c_0, ind_weights,
        ↪ hidden_weights, cell_weights, res_len, ind_act_typ,
        ↪ res_act_typ)
```

```

ctx.save_for_backward(y, all_h, all_c, ind_weights,
    ↪ hidden_weights, cell_weights)
ctx.saved_nontensors = (res_len, ind_act_typ, res_act_typ)
return out

@staticmethod
def backward(ctx, dout: Tensor) -> Tuple[Tensor, Tensor, Tensor,
    ↪ Tensor, Tensor, Tensor]:
    y, all_h, all_c, ind_weights, hidden_weights, cell_weights =
        ↪ ctx.saved_tensors
    res_len, ind_act_typ, res_act_typ = ctx.saved_nontensors
    dy, dh_t, dc_t, dind_weights, dhidden_weights, dcell_weights =
        ↪ indrevrnn_py_bwd(dout, y, all_h, all_c, ind_weights,
        ↪ hidden_weights, cell_weights, res_len, ind_act_typ,
        ↪ res_act_typ)
    return dy, dh_t, dc_t, dind_weights, dhidden_weights,
        ↪ dcell_weights, None, None, None

```

The meaning of each parameter is listed in the comments.

```

# y: Tensor
# input of IndRevRNN
# in the experiments, the input of IndRevRNN is the output of a fully
    ↪ connected layer
#  $y = W * x + b$ , while  $x$  denotes the initial input, which is task
    ↪ relevant

# h_0: Tensor
# initialized hidden states

# c_0: Tensor
# initialized cell states

# ind_weights (independently weight vector): Tensor
# weights on the hidden states before reversible residual blocks

# hidden_weights (hidden weight vector): Tensor
# weights on the branch with hidden states as input
# hidden_weights[i] denotes the hidden weight vector of the  $i$ -th
    ↪ reversible residual block

```

```

# cell_weights (cell weight vector): Tensor
# weights on the branch with cell states as input
# cell_weights[i] denotes the cell weight vector of the i-th reversible
↪ residual block

# res_len (reversible residual block length): int
# determines how many blocks does the reversible residual loop have

# ind_act_typ (independently activation type): int
# determines which activation function in the independently part is used
# ind_act_typ = 0: relu, ind_act_typ = 1: tanh, ind_act_typ = 2: sigmoid

# res_act_typ (reversible residual activation type): int
# determines which activation function in the reversible residual blocks
↪ is used
# res_act_typ = 0: relu, res_act_typ = 1: tanh, res_act_typ = 2: sigmoid

# out: Tensor
# output of the IndRevRNN forward function

# all_h (all hidden states): Tensor
# a tensor containing all hidden states at each time step of IndRevRNN
# all_h[t] denotes the hidden state tensor at time step t

# all_c (all cell states): Tensor
# a tensor containing all cell states at each time step of IndRevRNN
# all_c[t] denotes the cell state tensor at time step t

# variables in the backward function correspond to the variables in the
↪ forward function

```

In the above program, `ctx.save_for_backward` and `ctx.saved_nontensors` are responsible for passing variables from the forward function to the backward function.

Before implementing two functions `indrevrnn_py_fwd` and `indrevrnn_py_bwd` that are related to computations, some supplements need to be made to the activation functions.

The forward pass of activation functions such as ReLU, tanh, and Sigmoid has been implemented by Pytorch. To customize the backward functions in a neural network module, it is necessary to manually implement the derivatives of these functions and bind

them.

```
@unique
class Activation(int, Enum):
    relu = 0
    tanh = 1
    sigmoid = 2

def _d_relu(t):
    return (t >= 0).type_as(t)

def _d_tanh(t):
    tt = torch.tanh(t)
    return 1 - tt * tt

def _d_sigmoid(t):
    tt = torch.sigmoid(t)
    return tt * (1 - tt)

act_func = [(torch.relu, _d_relu), (torch.tanh, _d_tanh), (torch.sigmoid,
↪ _d_sigmoid)]
```

The following program implements the forward pass of IndRevRNN, i.e., the computation of Equation 4.23 to Equation 4.29. `seq_len` is the total number of time steps in the sequence, determined by the input. `ind_act` and `res_act` are the above-mentioned forward activation functions.

```
def indrevrnn_py_fwd(y: Tensor, h_0: Tensor, c_0: Tensor, ind_weights:
↪ Tensor, hidden_weights: Tensor, cell_weights: Tensor, res_len: int,
↪ ind_act_typ: int, res_act_typ: int) -> Tuple[Tensor, Tensor, Tensor]:
    all_h = torch.cat([h_0[None], torch.empty_like(y)], 0)
    all_c = torch.cat([c_0[None], torch.empty_like(y)], 0)
    out = torch.zeros_like(y)
    seq_len = y.size(0)
    ind_act = act_func[ind_act_typ][0]
    res_act = act_func[res_act_typ][0]
    for t in range(seq_len):
        h_t = all_h[t]
        c_t = all_c[t]
```

```

h_t = ind_act(y[t] + h_t)
out[t] = h_t
h_t = ind_weights * h_t
for i in range(res_len):
    c_t = c_t + hidden_weights[i] * res_act(h_t)
    h_t = h_t + cell_weights[i] * res_act(c_t)
all_h[t + 1] = h_t
all_c[t + 1] = c_t
return out, all_h, all_c

```

The following program implements the backward pass of IndRevRNN, i.e., the computation of Equation 4.30 to Equation 4.43. `ind_back` and `res_back` are the above-mentioned backward activation functions.

```

def indrevrnn_py_bwd(dout: Tensor, y: Tensor, all_h: Tensor, all_c:
    ↪ Tensor, ind_weights: Tensor, hidden_weights: Tensor, cell_weights:
    ↪ Tensor, res_len: int, ind_act_typ: int, res_act_typ: int) ->
    ↪ Tuple[Tensor, Tensor, Tensor, Tensor, Tensor, Tensor]:
    dy = torch.empty_like(y)
    dh_t = torch.zeros_like(all_h[0])
    dc_t = torch.zeros_like(all_c[0])
    dind_weights = torch.zeros_like(ind_weights)
    dhidden_weights = torch.zeros_like(hidden_weights)
    dcell_weights = torch.zeros_like(cell_weights)
    ind_act = act_func[ind_act_typ][0]
    res_act = act_func[res_act_typ][0]
    ind_back = act_func[ind_act_typ][1]
    res_back = act_func[res_act_typ][1]
    seq_len = y.size(0)
    for t in range(seq_len - 1, -1, -1):
        h_t = all_h[t + 1]
        c_t = all_c[t + 1]
        for i in range(res_len - 1, -1, -1):
            dcell_weights[i] = dcell_weights[i] + torch.sum(dh_t *
                ↪ res_act(c_t), 0)
            dc_t = dc_t + dh_t * cell_weights[i] * res_back(c_t)
            h_t = h_t - cell_weights[i] * res_act(c_t)
            dhidden_weights[i] = dhidden_weights[i] + torch.sum(dc_t *
                ↪ res_act(h_t), 0)
            dh_t = dh_t + dc_t * hidden_weights[i] * res_back(h_t)

```

```

        c_t = c_t - hidden_weights[i] * res_act(h_t)
    dind_weights = dind_weights + torch.sum(dh_t * ind_act(y[t] +
    ↪ all_h[t]), 0)
    dh_t = dh_t * ind_weights
    dh_t = dh_t + dout[t]
    dy[t] = dh_t * ind_back(y[t] + all_h[t])
    dh_t = dy[t]
    return dy, dh_t, dc_t, dind_weights, dhidden_weights, dcell_weights

```

So far, the Python implementation of IndRevRNN is complete.

A.2 Custom CUDA implementation of IndRevRNN

The program in the previous section does the full functionality of IndRevRNN. However, in the above tensor computation, PyTorch has to execute operations individually, one after the other. Frequent calls to operations involving launching CUDA kernels also have some overhead. Moreover, the Python interpreter running the code itself may slow down the program [1].

These problems make the execution of the above program very inefficient. In fact, as described by chapter 4, the independency in IndRevRNN allows many computations to be done in parallel. This will take advantage of the CUDA extension provided by Pytorch, which profits from fewer kernel launches as well as other optimizations that can be performed with increased visibility of the global flow of data [1].

To use the CUDA extension, a class similar to `_IndRevRNNTFuncPy` in the previous section is defined, as shown below.

```

class _IndRevRNNTFuncCUDA(Function):
    @staticmethod
    def forward(ctx, y: Tensor, h_0: Tensor, c_0: Tensor, ind_weights:
    ↪ Tensor, hidden_weights: Tensor, cell_weights: Tensor, res_len:
    ↪ int, ind_act_typ: int, res_act_typ: int) -> Tensor:
        out, all_h, all_c = indrevrnn_cuda.forward(y, h_0, c_0,
        ↪ ind_weights, hidden_weights, cell_weights, res_len,
        ↪ ind_act_typ, res_act_typ)
        ctx.save_for_backward(y, all_h, all_c, ind_weights,
        ↪ hidden_weights, cell_weights)
        ctx.saved_nontensors = (res_len, ind_act_typ, res_act_typ)
    return out

```

```

@staticmethod
def backward(ctx, dout: Tensor) -> Tuple[Tensor, Tensor, Tensor,
    ↪ Tensor, Tensor, Tensor]:
    y, all_h, all_c, ind_weights, hidden_weights, cell_weights =
    ↪ ctx.saved_tensors
    res_len, ind_act_typ, res_act_typ = ctx.saved_nontensors
    dy, dh_t, dc_t, dind_weights, dhidden_weights, dcell_weights =
    ↪ indrevrnn_cuda.backward(dout, y, all_h, all_c, ind_weights,
    ↪ hidden_weights, cell_weights, res_len, ind_act_typ,
    ↪ res_act_typ)
    return dy, dh_t, dc_t, dind_weights, dhidden_weights,
    ↪ dcell_weights, None, None, None

```

In order to bind the written CUDA extension with other programs in Python, one needs to define a `setup.py` and add the following content to it.

```

from setuptools import setup
from torch.utils.cpp_extension import (BuildExtension, CUDAExtension)

setup(
    name='torch-indrevrnn',
    ext_modules=[
        CUDAExtension('torch_indrevrnn.indrevrnn_cuda', [
            'torch_indrevrnn/indrevrnn_cuda.cpp',
            'torch_indrevrnn/indrevrnn_cuda_kernel.cu']),
    ],
    cmdclass={
        'build_ext': BuildExtension
    })

```

The first strategy for writing CUDA extensions is to write a C++ file that defines functions to be called from Python, and use pybind11 to bind those functions to Python. Additionally, this file will declare functions defined in CUDA (.cu) files. The C++ function will then do some checks and finally forward its calls to the CUDA functions [1]. Below is the C++ function of IndRevRNN.

```

#include <torch/extension.h>
#include <vector>
#include <stdio.h>

```

```
// CUDA forward declarations

std::vector<torch::Tensor> indrevrnn_cuda_forward(
    torch::Tensor y,
    torch::Tensor h_0,
    torch::Tensor c_0,
    torch::Tensor ind_weights,
    torch::Tensor hidden_weights,
    torch::Tensor cell_weights,
    int res_len,
    int ind_act_typ,
    int res_act_typ);

std::vector<torch::Tensor> indrevrnn_cuda_backward(
    torch::Tensor dout,
    torch::Tensor y,
    torch::Tensor all_h,
    torch::Tensor all_c,
    torch::Tensor ind_weights,
    torch::Tensor hidden_weights,
    torch::Tensor cell_weights,
    int res_len,
    int ind_act_typ,
    int res_act_typ);

// C++ interface

#define CHECK_CUDA(x) AT_ASSERTM(x.type().is_cuda(), #x " must be a CUDA  

↪ tensor")
#define CHECK_CONTIGUOUS(x) AT_ASSERTM(x.is_contiguous(), #x " must be  

↪ contiguous")
#define CHECK_INPUT(x) CHECK_CUDA(x); CHECK_CONTIGUOUS(x)

std::vector<torch::Tensor> indrevrnn_forward(
    torch::Tensor y,
    torch::Tensor h_0,
    torch::Tensor c_0,
    torch::Tensor ind_weights,
    torch::Tensor hidden_weights,
```

```
    torch::Tensor cell_weights,
    int res_len,
    int ind_act_typ,
    int res_act_typ)
{
    CHECK_INPUT(y);
    CHECK_INPUT(h_0);
    CHECK_INPUT(c_0);
    CHECK_INPUT(ind_weights);
    CHECK_INPUT(hidden_weights);
    CHECK_INPUT(cell_weights);

    return indrevrnn_cuda_forward(y, h_0, c_0, ind_weights,
↪ hidden_weights, cell_weights, res_len, ind_act_typ, res_act_typ);
}

std::vector<torch::Tensor> indrevrnn_backward(
    torch::Tensor dout,
    torch::Tensor y,
    torch::Tensor all_h,
    torch::Tensor all_c,
    torch::Tensor ind_weights,
    torch::Tensor hidden_weights,
    torch::Tensor cell_weights,
    int res_len,
    int ind_act_typ,
    int res_act_typ)
{
    CHECK_INPUT(dout);
    CHECK_INPUT(y);
    CHECK_INPUT(all_h);
    CHECK_INPUT(all_c);
    CHECK_INPUT(ind_weights);
    CHECK_INPUT(hidden_weights);
    CHECK_INPUT(cell_weights);
    return indrevrnn_cuda_backward(dout, y, all_h, all_c, ind_weights,
↪ hidden_weights, cell_weights, res_len, ind_act_typ, res_act_typ);
}
```

```
PYBIND11_MODULE(TORCH_EXTENSION_NAME, m)
{
    m.def("forward", &indrevrnn_forward, "indrevrnn forward (CUDA)");
    m.def("backward", &indrevrnn_backward, "indrevrnn backward (CUDA)");
}
```

This C++ function mainly plays the role of checking and forwarding functions defined in CUDA files to Python programs. Similar to the Python part, the forward and backward activation functions of the CUDA version need to be manually implemented, stored here in a file named `cuda_activation.cuh`.

```
#pragma once

#include <torch/extension.h>
#include <cuda.h>
#include <cuda_runtime.h>
#include <stdio.h>

template <typename scalar_t>
__device__ static scalar_t forward_relu(scalar_t x)
{
    return (x > 0.0) ? x : 0.0;
}

template <typename scalar_t>
__device__ static scalar_t backward_relu(scalar_t x)
{
    return (x > 0.0) ? 1.0 : 0.0;
}

template <typename scalar_t>
__device__ static scalar_t forward_tanh(scalar_t x)
{
    return tanh(x);
}

template <typename scalar_t>
__device__ static scalar_t backward_tanh(scalar_t x)
{

```

```
    const auto t = tanh(x);
    return 1 - (t * t);
}

template <typename scalar_t>
__device__ static scalar_t forward_sigmoid(scalar_t x)
{
    return 1 / (1 + exp(-x));
}

template <typename scalar_t>
__device__ static scalar_t backward_sigmoid(scalar_t x)
{
    const auto t = 1 / (1 + exp(-x));
    return t * (1 - t);
}

template <typename scalar_t>
__device__ static scalar_t cuda_forward(scalar_t x, int act_typ)
{
    switch (act_typ) {
        case 0:
            return forward_relu(x);
        case 1:
            return forward_tanh(x);
        case 2:
            return forward_sigmoid(x);
        default:
            printf("Unknown activation function type");
    }
}

template <typename scalar_t>
__device__ static scalar_t cuda_backward(scalar_t x, int act_typ)
{
    switch (act_typ) {
        case 0:
            return backward_relu(x);
        case 1:
```

```

        return backward_tanh(x);
    case 2:
        return backward_sigmoid(x);
    default:
        printf("Unknown activation function type");
    }
}

```

Next is the CUDA (.cu) file for IndRevRNN, starting with CUDA host functions that wrap the CUDA kernels to be accelerated.

```

#include <torch/extension.h>
#include <cuda.h>
#include <cuda_runtime.h>
#include <vector>
#include <stdio.h>
#include "cuda_activation.cuh"

// cuda host functions for IndRevRNN

std::vector<torch::Tensor> indrevrnn_cuda_forward(
    torch::Tensor y,
    torch::Tensor h_0,
    torch::Tensor c_0,
    torch::Tensor ind_weights,
    torch::Tensor hidden_weights,
    torch::Tensor cell_weights,
    int res_len,
    int ind_act_typ,
    int res_act_typ)
{
    const auto batch_size = y.size(1);
    const auto hidden_size = y.size(2);
    const int threads = 1024;
    const int blocks = (batch_size * hidden_size + threads - 1) /
→ threads;
    auto all_h = torch::cat({ torch::unsqueeze(h_0, 0),
→ torch::empty_like(y) }, 0);
    auto all_c = torch::cat({ torch::unsqueeze(c_0, 0),
→ torch::empty_like(y) }, 0);

```

```

auto out = torch::zeros_like(y);
int seq_len = y.size(0);
AT_DISPATCH_FLOATING_TYPES(y.type(), "indrevrnn_forward", ([&] {
    indrevrnn_forward_kernel<scalar_t> << <blocks, threads >> > (
        out.packed_accessor32<scalar_t, 3>(),
        y.packed_accessor32<scalar_t, 3>(),
        all_h.packed_accessor32<scalar_t, 3>(),
        all_c.packed_accessor32<scalar_t, 3>(),
        ind_weights.packed_accessor32<scalar_t, 2>(),
        hidden_weights.packed_accessor32<scalar_t, 2>(),
        cell_weights.packed_accessor32<scalar_t, 2>(),
        seq_len,
        res_len,
        ind_act_typ,
        res_act_typ);
}));
return { out, all_h, all_c };
}

std::vector<torch::Tensor> indrevrnn_cuda_backward(
    torch::Tensor dout,
    torch::Tensor y,
    torch::Tensor all_h,
    torch::Tensor all_c,
    torch::Tensor ind_weights,
    torch::Tensor hidden_weights,
    torch::Tensor cell_weights,
    int res_len,
    int ind_act_typ,
    int res_act_typ)
{
    const auto batch_size = y.size(1);
    const auto hidden_size = y.size(2);
    const int threads = 1024;
    const int blocks = (batch_size * hidden_size + threads - 1) /
    ↪ threads;
    auto dy = torch::zeros_like(y);
    auto dh_t = torch::zeros_like(all_h[0]);
    auto dc_t = torch::zeros_like(all_c[0]);

```

```

    auto dind_weights = torch::zeros_like(ind_weights).repeat({
↪ batch_size, 1, 1 });
    auto dhidden_weights = torch::zeros_like(hidden_weights).repeat({
↪ batch_size, 1, 1 });
    auto dcell_weights = torch::zeros_like(cell_weights).repeat({
↪ batch_size, 1, 1 });
    int seq_len = y.size(0);
    AT_DISPATCH_FLOATING_TYPES(y.type(), "indrevrnn_backward", ([&] {
        indrevrnn_backward_kernel<scalar_t> << <blocks, threads >> > (
            dout.packed_accessor32<scalar_t, 3>(),
            y.packed_accessor32<scalar_t, 3>(),
            all_h.packed_accessor32<scalar_t, 3>(),
            all_c.packed_accessor32<scalar_t, 3>(),
            ind_weights.packed_accessor32<scalar_t, 2>(),
            hidden_weights.packed_accessor32<scalar_t, 2>(),
            cell_weights.packed_accessor32<scalar_t, 2>(),
            dy.packed_accessor32<scalar_t, 3>(),
            dh_t.packed_accessor32<scalar_t, 2>(),
            dc_t.packed_accessor32<scalar_t, 2>(),
            dind_weights.packed_accessor32<scalar_t, 3>(),
            dhidden_weights.packed_accessor32<scalar_t, 3>(),
            dcell_weights.packed_accessor32<scalar_t, 3>(),
            seq_len,
            res_len,
            ind_act_typ,
            res_act_typ);
    }));
    auto dind_weights_sum = torch::sum(dind_weights, 0);
    auto dhidden_weights_sum = torch::sum(dhidden_weights, 0);
    auto dcell_weights_sum = torch::sum(dcell_weights, 0);
    return { dy, dh_t, dc_t, dind_weights_sum, dhidden_weights_sum,
↪ dcell_weights_sum };
}

```

The CUDA kernels for computations in the forward and backward pass of IndRevRNN are shown below:

```
namespace {
```

```

// forward kernel for the external IndRevRNN
template <typename scalar_t>
__launch_bounds__(1024)
__global__ void indrevrnn_forward_kernel(
    torch::PackedTensorAccessor32<scalar_t, 3> out,
    torch::PackedTensorAccessor32<scalar_t, 3> y,
    torch::PackedTensorAccessor32<scalar_t, 3> all_h,
    torch::PackedTensorAccessor32<scalar_t, 3> all_c,
    torch::PackedTensorAccessor32<scalar_t, 2> ind_weights,
    torch::PackedTensorAccessor32<scalar_t, 2> hidden_weights,
    torch::PackedTensorAccessor32<scalar_t, 2> cell_weights,
    int seq_len,
    int res_len,
    int ind_act_typ,
    int res_act_typ)
{
    const auto batch_size = y.size(1);
    const auto hidden_size = y.size(2);
    // column index
    const int c = blockIdx.x * blockDim.x + threadIdx.x;
    // return if out of range
    if (c >= batch_size * hidden_size) return;
    // get current batch and state index numbers
    const int bs = c / hidden_size;
    const int idx = c % hidden_size;
    // run indrevrnn for this thread
    for (int t = 0; t < seq_len; ++t)
    {
        scalar_t h_t = all_h[t][bs][idx];
        scalar_t c_t = all_c[t][bs][idx];
        h_t = cuda_forward(y[t][bs][idx] + h_t, ind_act_typ);
        out[t][bs][idx] = h_t;
        h_t = ind_weights[0][idx] * h_t;
        for (int i = 0; i < res_len; ++i)
        {
            c_t = c_t + hidden_weights[i][idx] * cuda_forward(h_t,
↪ res_act_typ);
            h_t = h_t + cell_weights[i][idx] * cuda_forward(c_t,
↪ res_act_typ);
        }
    }
}

```

```

    }
    all_h[t + 1][bs][idx] = h_t;
    all_c[t + 1][bs][idx] = c_t;
}
}

// backward kernel for the external IndRevRNN
template <typename scalar_t>
__launch_bounds__(1024)
__global__ void indrevrnn_backward_kernel(
    torch::PackedTensorAccessor32<scalar_t, 3> dout,
    torch::PackedTensorAccessor32<scalar_t, 3> y,
    torch::PackedTensorAccessor32<scalar_t, 3> all_h,
    torch::PackedTensorAccessor32<scalar_t, 3> all_c,
    torch::PackedTensorAccessor32<scalar_t, 2> ind_weights,
    torch::PackedTensorAccessor32<scalar_t, 2> hidden_weights,
    torch::PackedTensorAccessor32<scalar_t, 2> cell_weights,
    torch::PackedTensorAccessor32<scalar_t, 3> dy,
    torch::PackedTensorAccessor32<scalar_t, 2> dh_t,
    torch::PackedTensorAccessor32<scalar_t, 2> dc_t,
    torch::PackedTensorAccessor32<scalar_t, 3> dind_weights,
    torch::PackedTensorAccessor32<scalar_t, 3> dhidden_weights,
    torch::PackedTensorAccessor32<scalar_t, 3> dcell_weights,
    int seq_len,
    int res_len,
    int ind_act_typ,
    int res_act_typ)
{
    const auto batch_size = y.size(1);
    const auto hidden_size = y.size(2);
    // column index
    const int c = blockIdx.x * blockDim.x + threadIdx.x;
    // return if out of range
    if (c >= batch_size * hidden_size) return;
    // get current batch and state index numbers
    const int bs = c / hidden_size;
    const int idx = c % hidden_size;
    // run indrevrnn for this thread
    for (int t = seq_len - 1; t > -1; --t)

```

```

{
    scalar_t h_t = all_h[t + 1][bs][idx];
    scalar_t c_t = all_c[t + 1][bs][idx];
    for (int i = res_len - 1; i > -1; --i)
    {
        dcell_weights[bs][i][idx] = dcell_weights[bs][i][idx] +
↪ dh_t[bs][idx] * cuda_forward(c_t, res_act_typ);
        dc_t[bs][idx] = dc_t[bs][idx] + dh_t[bs][idx] *
↪ cell_weights[i][idx] * cuda_backward(c_t, res_act_typ);
        h_t = h_t - cell_weights[i][idx] * cuda_forward(c_t,
↪ res_act_typ);
        dhidden_weights[bs][i][idx] = dhidden_weights[bs][i][idx]
↪ + dc_t[bs][idx] * cuda_forward(h_t, res_act_typ);
        dh_t[bs][idx] = dh_t[bs][idx] + dc_t[bs][idx] *
↪ hidden_weights[i][idx] * cuda_backward(h_t, res_act_typ);
        c_t = c_t - hidden_weights[i][idx] * cuda_forward(h_t,
↪ res_act_typ);
    }
    dind_weights[bs][0][idx] = dind_weights[bs][0][idx] +
↪ dh_t[bs][idx] * cuda_forward(y[t][bs][idx] + all_h[t][bs][idx],
↪ ind_act_typ);
    dh_t[bs][idx] = dh_t[bs][idx] * ind_weights[0][idx];
    dh_t[bs][idx] = dh_t[bs][idx] + dout[t][bs][idx];
    dy[t][bs][idx] = dh_t[bs][idx] * cuda_backward(y[t][bs][idx] +
↪ all_h[t][bs][idx], ind_act_typ);
    dh_t[bs][idx] = dy[t][bs][idx];
}
}

} // namespace

```

It can be seen from the above kernel functions that the dimensions of each tensor, namely the time step [t] or the block index [i], the batch number [bs] and the element index [idx], are all involved in the computation in parallel. This allows the kernel functions to run the first line of the next inner or outer loop without waiting for the previous one to finish executing, which greatly improves the efficiency of the algorithm.

So far, one only needs to run the following code to complete the compilation of the CUDA extension for the IndRevRNN algorithm.

```
python setup.py build develop
```

A.3 IndRevRNN class module and parameter initialization

The following part shows the initialization of the hidden state vector $\mathbf{h}_0$ and cell state vector $\mathbf{c}_0$ in the IndRevRNN network without external specification and how to flatten the input vector $\mathbf{y}$ when it contains more than one feature dimension. It also shows how to call the IndRevRNN functions implemented in the previous two sections.

```
# nn.functional style implementation of IndRevRNN for tensor
# y (flattened): shape (seq_len, batch_size, hidden_size);
# ind_weights: shape (1, hidden_size);
# hidden_weights: shape (block_length, hidden_size);
# cell_weights: shape (block_length, hidden_size);
# (If specified) h_0: shape (batch_size, hidden_size) c_0: shape
↪ (batch_size, hidden_size).
# if y has more than one feature dimension, use flatten=True

def _indrevrnn(y: Tensor, h_0: Optional[Tensor], c_0: Optional[Tensor],
↪ ind_weights: Tensor, hidden_weights: Tensor, cell_weights: Tensor,
↪ res_len: int, ind_act_typ: int, res_act_typ: int, flatten: bool =
↪ False) -> Tensor:
    if h_0 is None:
        h_0 = torch.zeros_like(y[0])
    if c_0 is None:
        c_0 = torch.zeros_like(y[0])
    if flatten:
        yshape = y.size()
        y = y.view(*yshape[:2], -1)
        h_0 = h_0.view(h_0.size(0), -1)
        c_0 = c_0.view(c_0.size(0), -1)
    if y.is_cuda:
        out = _IndRevRNNFuncCUDA.apply(y, h_0, c_0, ind_weights,
↪ hidden_weights, cell_weights, res_len, ind_act_typ,
↪ res_act_typ)
    else:
        out = _IndRevRNNFuncPy.apply(y, h_0, c_0, ind_weights,
↪ hidden_weights, cell_weights, res_len, ind_act_typ,
↪ res_act_typ)
    return out.view(yshape) if flatten else out
```

```
# nn.functional style implementation of IndRevRNN that handles Tensor
def indrevrnn(y: Tensor, h_0: Optional[Tensor], c_0: Optional[Tensor],
    ↪ ind_weights: Tensor, hidden_weights: Tensor, cell_weights: Tensor,
    ↪ res_len: int, ind_act_typ: int, res_act_typ: int, flatten: bool =
    ↪ False) -> Tensor:
    if isinstance(y, Tensor):
        return _indrevrnn(y, h_0, c_0, ind_weights, hidden_weights,
            ↪ cell_weights, res_len, ind_act_typ, res_act_typ, flatten)
    else:
        raise TypeError('y should be of type torch.Tensor, '
            'found {}'.format(type(y)))
```

The following shows the class that wraps the IndRevRNN into a neural network module, and how to initialize the weight vectors in the network.

```
# IndRevRNN module, to be used together with e.g., Linear or Conv*d
class IndRevRNN(nn.Module):
    def __init__(self, seq_len: int, hidden_size: int, res_len: int,
    ↪ ind_act_typ: int, res_act_typ: int):
        assert hidden_size > 0
        super(IndRevRNN, self).__init__()
        self.seq_len = seq_len
        self.hidden_size = hidden_size
        self.res_len = res_len
        self.ind_act_typ = ind_act_typ
        self.res_act_typ = res_act_typ
        self.register_parameter(name="ind_weights",
            ↪ param=nn.Parameter(torch.empty(1, hidden_size),
            ↪ requires_grad=True))
        self.register_parameter(name="hidden_weights",
            ↪ param=nn.Parameter(torch.empty(self.res_len, hidden_size),
            ↪ requires_grad=True))
        self.register_parameter(name="cell_weights",
            ↪ param=nn.Parameter(torch.empty(self.res_len, hidden_size),
            ↪ requires_grad=True))
        if torch.cuda.is_available():
            self.ind_weights.cuda()
            self.hidden_weights.cuda()
            self.cell_weights.cuda()
```

```
self.reset_parameters()

def reset_parameters(self) -> None:
    nn.init.normal_(self.hidden_weights, mean=0.0, std=1e-8)
    nn.init.normal_(self.cell_weights, mean=0.0, std=1e-8)
    nn.init.uniform_(self.ind_weights, a=0, b=1)

def forward(self, y: Tensor, h_0: Optional[Tensor] = None,
            c_0: Optional[Tensor] = None, flatten: bool = False) ->
    ↪ Tensor:
    """y (flattened): shape (seq_len, batch_size, hidden_size).
    If specified, h_0: shape (batch_size, hidden_size), c_0: shape
    ↪ (batch_size, hidden_size).
    If y has more than one feature dimensions, use flatten=True."""
    return indrevrnn(y, h_0, c_0, self.ind_weights,
    ↪ self.hidden_weights, self.cell_weights, self.res_len,
    ↪ self.ind_act_typ, self.res_act_typ, flatten)
```

List of Figures

1.1	Basic structure and recurrent units of an RNN.	1
2.1	The LSTM network.	5
2.2	The GRU cell.	6
2.3	The original dropout scheme.	8
2.4	First dropout applied to RNNs.	9
2.5	Dropout examples in an LSTM cell.	10
2.6	Dimensions of Batch Normalization, Layer Normalization and Group Normalization in an RNN.	12
3.1	A typical vanilla RNN structure.	13
3.2	The vanilla RNN cell.	14
3.3	Connections of independent neurons in IndRNN.	16
3.4	A reversible residual block inspired by RevNet.	18
4.1	The structure of IndLSTM.	21
4.2	Basic IndRevRNN.	24
4.3	IndRevRNN with independently weight vector before reversible residual blocks.	27
4.4	IndRevRNN with post residual activation functions.	27
4.5	IndRevRNN with residual cell factors.	28
4.6	IndRevRNN with sequence cell factors.	29
4.7	Deformed variants of cell factors.	30
4.8	IndRevRNN with forget gate from input.	31
4.9	IndRevRNN with forget gate from before recurrent.	31
4.10	IndRevRNN with forget gate from after activation.	32
5.1	Three groups of parameters to initialize in the basic IndRevRNN.	35
5.2	Images in the MNIST dataset.	37
5.3	Images in the permuted sequential MNIST dataset in a 2D space.	38
5.4	Model architecture used for experiments on the permuted sequential MNIST dataset.	39
5.5	Example sentences in PTB.	42

5.6	Model architecture used for experiments on the PTB corpus.	44
-----	--	----

List of Tables

5.1	Initialization of model parameters.	36
5.2	Average training time per epoch of different independently recurrent networks on the permuted sequential MNIST dataset.	40
5.3	Average test accuracies of different IndRevRNN variants on the permuted sequential MNIST dataset.	41
5.4	Test accuracies of IndRevRNN and state-of-the-art RNNs on the permuted sequential MNIST dataset.	42
5.5	Average training time per epoch of IndRevRNNs with different block lengths on the PTB corpus.	45
5.6	Average test perplexities of different IndRevRNN variants on the PTB corpus.	45
5.7	Test perplexities of IndRevRNN and state-of-the-art RNNs on the PTB corpus.	46

Bibliography

- [1] Custom c++ and cuda extensions. https://pytorch.org/tutorials/advanced/cpp_extension.html. Accessed: 2022-01-10.
- [2] Mohamed Abdel-Nasser and Karar Mahmoud. Accurate photovoltaic power forecasting models using deep lstm-rnn. *Neural Computing and Applications*, 31(7):2727–2740, 2019.
- [3] Alexandre Alahi, Kratarth Goel, Vignesh Ramanathan, Alexandre Robicquet, Li Fei-Fei, and Silvio Savarese. Social lstm: Human trajectory prediction in crowded spaces. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 961–971, 2016.
- [4] Saad Albawi, Tareq Abed Mohammed, and Saad Al-Zawi. Understanding of a convolutional neural network. In *2017 International Conference on Engineering and Technology (ICET)*, pages 1–6. Ieee, 2017.
- [5] D Anderson and K Burnham. Model selection and multi-model inference. *Second. NY: Springer-Verlag*, 63(2020):10, 2004.
- [6] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, Jorge Luis Reyes-Ortiz, et al. A public domain dataset for human activity recognition using smartphones. In *Esann*, volume 3, page 3, 2013.
- [7] Martin Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. In *International Conference on Machine Learning*, pages 1120–1128. PMLR, 2016.
- [8] Devansh Arpit, Bhargav Kanuparthi, Giancarlo Kerg, Nan Rosemary Ke, Ioannis Mitliagkas, and Yoshua Bengio. h-detach: Modifying the lstm gradient towards better optimization. *arXiv preprint arXiv:1810.03023*, 2018.
- [9] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [10] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

- [11] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [12] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. Trellis networks for sequence modeling. *arXiv preprint arXiv:1810.06682*, 2018.
- [13] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994.
- [14] James Bradbury, Stephen Merity, Caiming Xiong, and Richard Socher. Quasi-recurrent neural networks. *arXiv preprint arXiv:1611.01576*, 2016.
- [15] Víctor Campos, Brendan Jou, Xavier Giró-i Nieto, Jordi Torres, and Shih-Fu Chang. Skip rnn: Learning to skip state updates in recurrent neural networks. *arXiv preprint arXiv:1708.06834*, 2017.
- [16] Gang Chen. A gentle tutorial of recurrent neural network with error backpropagation. *arXiv preprint arXiv:1610.02583*, 2016.
- [17] Vinay Kumar Reddy Chimmula and Lei Zhang. Time series forecasting of covid-19 transmission in canada using lstm networks. *Chaos, Solitons & Fractals*, 135:109864, 2020.
- [18] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- [19] Tim Cooijmans, Nicolas Ballas, César Laurent, Çağlar Gülçehre, and Aaron Courville. Recurrent batch normalization. *arXiv preprint arXiv:1603.09025*, 2016.
- [20] Martin Cutts. *Oxford guide to plain English*. Oxford University Press, USA, 2020.
- [21] Andrew M Dai and Quoc V Le. Semi-supervised sequence learning. *Advances in neural information processing systems*, 28:3079–3087, 2015.
- [22] Rahul Dey and Fathi M Salem. Gate-variants of gated recurrent unit (gru) neural networks. In *2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS)*, pages 1597–1600. IEEE, 2017.
- [23] Marco Dinarelli and Loïc Grobol. Seq2biseq: Bidirectional output-wise recurrent neural networks for sequence modelling. *arXiv preprint arXiv:1904.04733*, 2019.

- [24] Brian S Everitt and Anders Skrondal. The cambridge dictionary of statistics. 2010.
- [25] Rui Fu, Zuo Zhang, and Li Li. Using lstm and gru neural network methods for traffic flow prediction. In *2016 31st Youth Academic Annual Conference of Chinese Association of Automation (YAC)*, pages 324–328. IEEE, 2016.
- [26] Yarín Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059. PMLR, 2016.
- [27] Felix A Gers and E Schmidhuber. Lstm recurrent networks learn simple context-free and context-sensitive languages. *IEEE Transactions on Neural Networks*, 12(6):1333–1340, 2001.
- [28] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feed-forward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- [29] Aidan N Gomez, Mengye Ren, Raquel Urtasun, and Roger B Grosse. The reversible residual network: Backpropagation without storing activations. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pages 2211–2221, 2017.
- [30] Ian J Goodfellow, Mehdi Mirza, Da Xiao, Aaron Courville, and Yoshua Bengio. An empirical investigation of catastrophic forgetting in gradient-based neural networks. *arXiv preprint arXiv:1312.6211*, 2013.
- [31] Klaus Greff, Rupesh K Srivastava, Jan Koutník, Bas R Steunebrink, and Jürgen Schmidhuber. Lstm: A search space odyssey. *IEEE transactions on neural networks and learning systems*, 28(10):2222–2232, 2016.
- [32] Nicole Gruber and Alfred Jockisch. Are gru cells more specific and lstm cells more sensitive in motive classification of text? *Frontiers in artificial intelligence*, 3:40, 2020.
- [33] Antonio Gulli and Sujit Pal. *Deep learning with Keras*. Packt Publishing Ltd, 2017.
- [34] Douglas M Hawkins. The problem of overfitting. *Journal of chemical information and computer sciences*, 44(1):1–12, 2004.
- [35] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.

- [36] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Identity mappings in deep residual networks. In *European conference on computer vision*, pages 630–645. Springer, 2016.
- [37] Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.
- [38] Sepp Hochreiter. The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(02):107–116, 1998.
- [39] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [40] Zhiheng Huang, Geoffrey Zweig, Michael Levit, Benoit Dumoulin, Barlas Oguz, and Shawn Chang. Accelerating recurrent neural network training via two stage classes and parallelization. In *2013 IEEE Workshop on Automatic Speech Recognition and Understanding*, pages 326–331. IEEE, 2013.
- [41] Heikki Hyötyniemi. Turing machines are recurrent neural networks. *Proceedings of step*, 96, 1996.
- [42] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015.
- [43] Rafal Jozefowicz, Wojciech Zaremba, and Ilya Sutskever. An empirical exploration of recurrent network architectures. In *International conference on machine learning*, pages 2342–2350. PMLR, 2015.
- [44] Dan Jurafsky. *Speech & language processing*. Pearson Education India, 2000.
- [45] Giancarlo Kerg, Kyle Goyette, Maximilian Puelma Touzel, Gauthier Gidel, Eugene Vorontsov, Yoshua Bengio, and Guillaume Lajoie. Non-normal recurrent neural network (nnrnn): learning long time dependencies while improving expressivity with transient dynamics. *arXiv preprint arXiv:1905.12080*, 2019.
- [46] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [47] Durk P Kingma, Tim Salimans, and Max Welling. Variational dropout and the local reparameterization trick. *Advances in neural information processing systems*, 28:2575–2583, 2015.

- [48] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of neural sequence models. In *International Conference on Machine Learning*, pages 2766–2775. PMLR, 2018.
- [49] David Krueger, Tegan Maharaj, János Kramár, Mohammad Pezeshki, Nicolas Ballas, Nan Rosemary Ke, Anirudh Goyal, Yoshua Bengio, Aaron Courville, and Chris Pal. Zoneout: Regularizing rnns by randomly preserving hidden activations. *arXiv preprint arXiv:1606.01305*, 2016.
- [50] Siddharth Krishna Kumar. On weight initialization in deep neural networks. *arXiv preprint arXiv:1704.08863*, 2017.
- [51] Gakuto Kurata, Bhuvana Ramabhadran, George Saon, and Abhinav Sethy. Language modeling with highway lstm. In *2017 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, pages 244–251. IEEE, 2017.
- [52] Ernst Kussul and Tatiana Baidyk. Improved method of handwritten digit recognition tested on mnist database. *Image and Vision Computing*, 22(12):971–981, 2004.
- [53] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [54] Tao Lei, Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Deriving neural architectures from sequence and graph kernels. In *International Conference on Machine Learning*, pages 2024–2033. PMLR, 2017.
- [55] Tao Lei, Yu Zhang, Sida I Wang, Hui Dai, and Yoav Artzi. Simple recurrent units for highly parallelizable recurrence. *arXiv preprint arXiv:1709.02755*, 2017.
- [56] Shuai Li, Wanqing Li, Chris Cook, and Yanbo Gao. Deep independently recurrent neural network (indrnn). *arXiv preprint arXiv:1910.06251*, 2019.
- [57] Shuai Li, Wanqing Li, Chris Cook, Ce Zhu, and Yanbo Gao. Independently recurrent neural network (indrnn): Building a longer and deeper rnn. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5457–5466, 2018.
- [58] Zachary C Lipton, John Berkowitz, and Charles Elkan. A critical review of recurrent neural networks for sequence learning. *arXiv preprint arXiv:1506.00019*, 2015.
- [59] David Luebke. Cuda: Scalable parallel programming for high-performance scientific computing. In *2008 5th IEEE international symposium on biomedical imaging: from nano to macro*, pages 836–838. IEEE, 2008.

- [60] Mitchell Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. 1993.
- [61] Stephen Merity, Nitish Shirish Keskar, and Richard Socher. Regularizing and optimizing lstm language models. *arXiv preprint arXiv:1708.02182*, 2017.
- [62] Yajie Miao, Mohammad Gowayyed, and Florian Metze. Eesen: End-to-end speech recognition using deep rnn models and wfst-based decoding. In *2015 IEEE Workshop on Automatic Speech Recognition and Understanding (ASRU)*, pages 167–174. IEEE, 2015.
- [63] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [64] Chigozie Nwankpa, Winifred Ijomah, Anthony Gachagan, and Stephen Marshall. Activation functions: Comparison of trends in practice and research for deep learning. *arXiv preprint arXiv:1811.03378*, 2018.
- [65] Christopher Olah. Understanding lstm networks. 2015.
- [66] Beresford Parlett. Laguerre’s method applied to the matrix eigenvalue problem. *Mathematics of Computation*, 18(87):464–485, 1964.
- [67] Dabal Pedomonti. Comparison of non-linear activation functions for deep neural networks on mnist classification task. *arXiv preprint arXiv:1804.02763*, 2018.
- [68] Mirco Ravanelli, Philemon Brakel, Maurizio Omologo, and Yoshua Bengio. Light gated recurrent units for speech recognition. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(2):92–102, 2018.
- [69] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
- [70] T Konstantin Rusch and Siddhartha Mishra. Coupled oscillatory recurrent neural network (cornn): An accurate and (gradient) stable architecture for learning long time dependencies. *arXiv preprint arXiv:2010.00951*, 2020.
- [71] Haşim Sak, Andrew Senior, and Françoise Beaufays. Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition. *arXiv preprint arXiv:1402.1128*, 2014.
- [72] Tim Salimans and Durk P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. *Advances in neural information processing systems*, 29:901–909, 2016.

- [73] Sreelekshmy Selvin, R Vinayakumar, EA Gopalakrishnan, Vijay Krishna Menon, and KP Soman. Stock price prediction using lstm, rnn and cnn-sliding window model. In *2017 international conference on advances in computing, communications and informatics (icacci)*, pages 1643–1647. IEEE, 2017.
- [74] Stanislau Semeniuta. *Recurrent Neural Networks for Discriminative and Generative Learning*. PhD thesis, Universität zu Lübeck, 2019.
- [75] Stanislau Semeniuta, Aliaksei Severyn, and Erhardt Barth. Recurrent dropout without memory loss. *arXiv preprint arXiv:1603.05118*, 2016.
- [76] Amir Shahroudy, Jun Liu, Tian-Tsong Ng, and Gang Wang. Ntu rgb+ d: A large scale dataset for 3d human activity analysis. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1010–1019, 2016.
- [77] Alex Sherstinsky. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, 404:132306, 2020.
- [78] Heng Shi, Minghao Xu, and Ran Li. Deep learning for household load forecasting—a novel pooling deep rnn. *IEEE Transactions on Smart Grid*, 9(5):5271–5280, 2017.
- [79] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- [80] Hendrik Strobelt, Sebastian Gehrmann, Hanspeter Pfister, and Alexander M Rush. Lstmvis: A tool for visual analysis of hidden state dynamics in recurrent neural networks. *IEEE transactions on visualization and computer graphics*, 24(1):667–676, 2017.
- [81] Martin Sundermeyer, Ralf Schlüter, and Hermann Ney. Lstm neural networks for language modeling. In *Thirteenth annual conference of the international speech communication association*, 2012.
- [82] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.
- [83] Jos Van Der Westhuizen and Joan Lasenby. The unreasonable effectiveness of the forget gate. *arXiv preprint arXiv:1804.04849*, 2018.
- [84] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.

- [85] Aaron R Voelker, Ivana Kajić, and Chris Eliasmith. Legendre memory units: Continuous-time representation in recurrent neural networks. 2019.
- [86] Sida Wang and Christopher Manning. Fast dropout training. In *international conference on machine learning*, pages 118–126. PMLR, 2013.
- [87] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.
- [88] Zhizheng Wu and Simon King. Investigating gated recurrent networks for speech synthesis. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5140–5144. IEEE, 2016.
- [89] Zhilin Yang, Zihang Dai, Ruslan Salakhutdinov, and William W Cohen. Breaking the softmax bottleneck: A high-rank rnn language model. *arXiv preprint arXiv:1711.03953*, 2017.
- [90] Xue Ying. An overview of overfitting and its solutions. In *Journal of Physics: Conference Series*, volume 1168, page 022022. IOP Publishing, 2019.
- [91] Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014.
- [92] Julian Georg Zilly, Rupesh Kumar Srivastava, Jan Koutník, and Jürgen Schmidhuber. Recurrent highway networks. In *International Conference on Machine Learning*, pages 4189–4198. PMLR, 2017.